RTEMS SMP

Ready to Fly

FV-201 Formal Verification Artefacts (Architecture, Models, Assumptions, Traceability, Supporting Tests)

Release 8

CONTENTS

1 Introduction **3**
1.1 The RTEMS SMP Project . . . 3
1.2 Formal Verification . . . 4
1.3 Summary of Outcome . . . 5
1.3.1 Achievements . . . 5
1.3.2 Unfinished Work . . . 5
1.3.3 Contents of `formal` directory . . . 5
1.4 Document Structure . . . 5

2 Methodology **7**
2.1 Methodology Introduction . . . 7
2.2 Promela/SPIN Overview . . . 7
2.2.1 Promela/SPIN . . . 8
2.3 Generating Tests from Promela/SPIN . . . 8

3 Tools **9**
3.1 Tools Introduction . . . 9
3.2 Spin2Test . . . 9
3.3 TestGen . . . 9
3.4 TestBuilder . . . 10

4 User Manual **11**
4.1 User Manual Introduction . . . 11
4.2 System Pre-Requisites . . . 11
4.3 Building Tools . . . 11
4.4 Running tools . . . 12
4.4.1 Spin2Test . . . 12
4.4.1.1 Steps to use *spin2test* to generate tests . . . 12
4.4.2 TestBuilder . . . 14
4.4.3 TestGen . . . 14
4.5 Manual Annotations . . . 14
4.5.1 Annotation Syntax . . . 15
4.5.2 Annotation Refinement . . . 15
4.6 Automatic Annotation Modelling . . . 16

5 Case Study 1: The Chains API **17**
5.1 Chain API Models (“Pure”) . . . 17

5.2 Annotated Models . . . 19
5.3 Manual Annotation Approach . . . 19
5.3.1 Manual Annotations . . . 20
5.3.2 Manual Refinement . . . 22

6 Case Study 2: The Event Manager **25**
6.1 Event Manager Model Overview . . . 26
6.2 Annotating Concurrency . . . 26

7 Case Study 3: MrsP Thread Queues **29**
7.1 Current State of Play . . . 29
7.1.1 Scoping . . . 29
7.1.2 Modelling . . . 29
7.1.3 Validation . . . 30
7.1.4 Verification . . . 30
7.2 Scoping Details . . . 30
7.3 Modelling Details . . . 31
7.3.1 SPARC TSO Model . . . 31
7.3.2 Configuration Generator Test . . . 31
7.3.3 MrsP Code Model . . . 32
7.3.3.1 Datatypes . . . 33
7.3.3.2 Queues . . . 33
7.3.3.3 Initialization . . . 34
7.3.3.4 Semaphore Operations . . . 34
7.3.4 MrsP Abstract Model . . . 35
7.4 Validation Details . . . 39
7.5 Verification Details . . . 39

8 Case Study 4: The Barrier Manager **41**
8.1 Manager Overview . . . 41
8.2 Model Overview . . . 41
8.3 Scenarios Overview . . . 42
8.4 Comparison with Event Manager . . . 42

9 Case Study 5: The Message Manager **43**
9.1 Manager Overview . . . 43
9.2 Model Overview . . . 43
9.3 Scenarios Overview . . . 44
9.4 Comparison with Event Manager . . . 44

10 APPENDIX **45**
10.1 MrsP Scenario 618 . . . 45
10.2 Description of *testgen* . . . 52
10.2.1 Specification of *testgen* . . . 52
10.2.1.1 Tool Usage . . . 52
10.2.1.2 Promela Annotations . . . 52
10.2.1.3 Refinement . . . 52
10.2.1.4 Documentation . . . 52
10.2.1.5 Constraints . . . 53
10.2.2 Implementation of *testgen* . . . 53

10.2.2.1 Structure of the Source . . . 53
10.2.2.2 Semantics of the Generation . . . 54
10.2.2.3 Extending Promela with an Annotation Language . . . 55
10.2.2.4 Semantics of Commands . . . 57
10.2.3 Generating a Trail File . . . 59
10.2.3.1 Main Commands . . . 59
10.2.3.2 Optional Commands . . . 60
10.2.4 Generating a Test File . . . 60
10.2.5 _ . . . 60
10.2.5.1 Main Commands . . . 60
10.2.6 Uniform Refinement . . . 60
10.2.6.1 Main Commands . . . 61
10.2.7 Generic Refinement . . . 62
10.2.7.1 Main Command . . . 62
10.2.8 Optional Commands . . . 63
10.2.9 Generating a Project Directory . . . 64
10.2.9.1 Main Command . . . 64
10.2.9.2 Optional Commands . . . 64
10.2.10 Checking Input Format . . . 65
10.2.10.1 Main Command . . . 65
10.2.11 Derivative Forms . . . 65
10.2.11.1 Positional Reference . . . 65
10.2.11.2 Content as File . . . 65
10.2.11.3 Positional Reference and Content as File . . . 66
10.3 Automatic Extraction of Promela Models from C11 Code . . . 66
10.3.1 Conversion Step Details . . . 68
10.3.1.1 C11 $\rightarrow$ $C^{PP}11$ . . . 68
10.3.1.2 $C^{PP}11 \rightarrow C^{PP}99_{annot11}$ . . . 68
10.3.1.3 $C^{PP}99_{annot11} \rightarrow C^{PP}99_{annot11}{}^{CIL}$ . . . 69
10.3.1.4 $C^{PP}99_{annot11}{}^{CIL} \rightarrow C^{PP}99_{annot11+CIL}{}^{ANSI}$ . . . 69
10.3.1.5 $C^{PP}99_{annot11+CIL}{}^{ANSI} \rightarrow PML^{PP}99_{annot11+CIL}{}^{ANSI}$ . . . 69
10.3.1.6 $C_{like}99/11/18^{PP} \rightarrow C_{like}99/11/18^{PP+TREE}$ . . . 70
10.3.2 Implementation and Statistics of the Sub-Generators . . . 70
10.3.2.1 A Many-Core Setting with 6 CPUs . . . 72
10.3.2.2 Same Configuration and Type of CPU as in Previous Experiments But Now With Only One Core . . . 74
10.3.2.3 Adjusted Elapsed Time Taking Into Consideration the Bandwidth Bitrates . . . 75
10.3.2.4 Synthesis of Resource Usage . . . 75
10.3.3 Conclusion . . . 77
10.4 Automatic Annotation Approach . . . 77
10.4.1 Model with Assertions . . . 78
10.4.2 Automatic Annotations . . . 79
10.4.3 “Automatic” Refinement . . . 80
10.4.4 Automatic Annotation Generation . . . 81
10.4.5 Automatic Counterexample Generation . . . 81
10.4.6 Automatic Code Generation . . . 82
10.4.7 “Automatic” Software Tools . . . 83
10.4.7.1 testgen . . . 83

10.5 Automatic Approach: Modelling of the Event Manager in Promela 84
10.5.1 Overview 84
10.5.2 Architecture Model 86

11 References **89**

Bibliography **91**

Identification, Copyrights and License

© 2019-2021 Trinity College Dublin

The copyright holders listed above grant that this document may be reproduced in whole or in part, or stored in a retrieval system, or transmitted in any form, or by any means electronic, mechanical, photocopying or otherwise, under the Creative Commons Attribution-ShareAlike 4.0 International Public License.

Release	Git hash	Date	Approved	Changes
01	55fd24df	Jun 30, 2020	Replaced	For DDR#1
02	8bf16481	Oct 19, 2020	Replaced	For DDR#2
03	18d02bb2	Mar 22, 2021	Replaced	For CDR
04	2ed87ffa	Jun 01, 2021	Replaced	For QR1
05	bd6115ce	Oct 07, 2021	Replaced	For QR2
06	c4cfa2d99	Nov 20, 2021	Replaced	Major re-write for QR2 closeout
07	dc258bbf2	Nov 29, 2021	Replaced	Clean up document
08	aa95122c	Mar 27, 2023	Approved	Reflected latest developments by Dr. Andrew and his team

	Name	Company	Signature
Written by	Andrew Butterfield, Frédéric Tuong	Trinity College Dublin	
Written by	ISVV Team	Critical Software	
Verified by	ISVV Team	Critical Software	
Approved by	ISVV Team	Critical Software	

	Name	Signature (if necessary)
Approved by customer (if necessary)		

CHAPTER

ONE

INTRODUCTION

1.1 The RTEMS SMP Project

The aim of the RTEMS SMP project is to pre-qualify RTEMS of the community (rtems.org) and to decrease the manual work required for future pre-qualifications. For this purpose the project is split into two main tasks:

- RTEMS pre-qualification
- Qualification Toolchain

The RTEMS pre-qualification involves the creation of the set of deliverables required by the relevant ECSS standards. RTEMS is an open-source RTOS created in the 1980s by the USA Defense Department that has since been made public. Today it is maintained by OAR. Hence, modifications to it (along with the new deliverables produced in this project) require authorization by OAR in order to be merged into the RTEMS mainline.

The aim of the Qualification toolchain is to generate outputs automatically (e.g. documentation) derived from a set of inputs. Some of these inputs are derived from the RTEMS ticket system while others are derived from the RTEMS repository.

In relation to the testing of RTEMS, there are two connected PCs running at ESTEC, one capable of building, loading and running executables on the GR712 and GR740 and another PC capable of building and running test executables on a SCOC3 machine with 1553 and UART connections to the boards. Access to some of this has been granted to consortium members.

In addition to the main two tasks above, there are two other tasks:

- Formal Verification
- Test Implementation

Test Implementation involves taking a real application running on a multiprocessing version of RTEMS, and performing pre-qualification on that, to assess the effectiveness of the proposed solutions.

This deliverable focusses on the Formal Verification task.

1.2 Formal Verification

This document is a Task 3 deliverable and describes the work done as part of Task 3.2. This task followed on from Task 3.1 [BH19] that involved an initial investigation into the formal techniques that we would adopt, as well as an initial discussion regarding what aspects of RTEMS would be verified.

Task 3.2 also saw the completion of the recruitment of a post-doctoral research fellow to assist with the work. Dr. Frédéric Tuong joined us from December 1st 2019 until February 12th 2021 to work on Task 3.2. He brought lot of expertise to the consortium, in Isabelle/HOL, Isabelle/C, and the OCaml package manager `opam`.

Initially, Task 3.2 continued further investigations into the following formal technologies and methodologies:

- Promela/SPIN
- Isabelle/C
- Frama-C

The key approach that was chosen was to focus on the use of Promela/SPIN to do modelling and test generation. This choice was based on two key factors: the fact that Promela/SPIN is a successful technology widely used in industry, and the fact that, as a software package, it has minimal pre-requistes and a small footprint.

Isabelle/C and Frama-C played a supporting role in helping us explore some ways of extracting Promela models from RTEMS code. Both technologies are much more niche in mature, and have complex dependencies and large footprints.

The decision on which RTEMS SMP algorithms to study was not taken at the end of Task 3.1/start of Task 3.2, but was delayed quite some time. Eventually it was decided that we would adopt a three-phase approach to modelling and test generation

1. The Chains API provides a key datastructure in RTEMS. We would model part of this in order to prototype the model/test generation methodology.
2. We would then model the Events Manager, to stress test the methodology and see how things would scale.
3. Finally, we would look at the most complex part of the scheduler, namely the thread queues associated with the MrsP semaphores.

In a second phase of this activity, the study of two new RTEMS SMP algorithms developed by Master students, both using Event Manager model as reference, were included:

1. The Barrier Manager, a barrier used for tasks synchronisation.
2. The Message Manager, message queue for Tasks communication.

From all five activities we would expect to see Promela model development coupled with python-based tool development, as required for this activity.

1.3 Summary of Outcome

1.3.1 Achievements

We have developed a framework for using Promela/SPIN to provide models of RTEMS behaviour, with tools to support the generation of tests from those models. We used the Chains API to prototype initial ideas, and then the Event Manager, Barrier Manager and Message Manager to explore issues pertaining to concurrency, priority, and multi-core. We have made progress in modelling the complex thread-queue system that is the core of a key multi-score scheduling algorithm in RTEMS.

1.3.2 Unfinished Work

The thread-queue needs more work. The simplest to achieve is getting tests generated from the high level model. More difficult is the code model. The issue here is how the MrsP thread queues are initialised, and will some help from an RTEMS scheduler domain expert. The process of modelling the algorithm code is straightforward, but checking it out and validating it is not possible until the initial state can be determined and modelled.

1.3.3 Contents of `formal` directory

At the top-level, the folders identity the main formal method in use. At present, there is only one: *promela*. This is structured as follows:

promela/models contains sub-folders for specific aspects of RTEMS:

- ***chains*** contains the Chains API model and test support files.
- ***events*** contains the Event Manager model and test support files.
- ***threadq*** contains the Thread Queue modelling, broken down into several sub-folders.
- ***barriers*** contains the Barriers Manager model and test support files.
- ***messages*** contains the Messages Manager model and test support files.

promela/src contains contains python source code and support

promela/proto contains prototype and exploratory material, in a number of sub-folders.

1.4 Document Structure

We first describe the overall methodology used for test generation from Promela models (Section 2).

We then describe the python-based tools developed to support test generation (Section 3).

Both the above sections provide a fairly high-level overview. All the fine gory details about how to find, setup and use the tools to support the methodoloy can be found in the user manual (Section 4).

We then report on five case studies, used to to develop the methods, tools and the corresponding tests. These case studies where: the Chain API (Section 5), the Event Manager (Section 6), the MrsP thread queues (Section 7), the Barrier Manager (Section 8), and the Message Manager (Section 9).

Some reference material has been relegated to the appendix (Section 10). Some of this older material that needs some updating, particularly as far as pathnames are concerned.

CHAPTER

TWO

METHODOLOGY

2.1 Methodology Introduction

The basic methodology for our work is to use a formal modelling language designed for model checking to build models of various aspects of RTEMS. We initially use the model-checker as normally intended, namely to ensure that the model has the correct properties as specified by assertions and other suitable logical statements. The model checker will build a transition state model which it will then exhaustively search to check correctness. If all is correct, it will affirm that this is the case. If it finds a violation of any specifed property, it will report it, and provide a *precise sequence of events* in the model that *lead to that violation*. Such a sequence is usually referred to a a *counterexample*. A fully correct model is one in which no sequence of events generated by the model violates any property

For test generation, given a *correct* model, what we do is to negate our statements of correct properties. Asking the model-checker to run again means that all event sequences it generates are correct, but our negated properties will be flagged as violation by the checker. These "counterexamples" are in fact traces of correct model behaviour. We can interpet these a testing scenarios.

We chose to do this using Promela/SPIN (spinroot.com)

2.2 Promela/SPIN Overview

The Promela language is a mixture of C and Dijkstra's guarded command language. It supports common C types, as well as arrays and structs, but does not have the notion of pointers. A Promela file typically has as `.pml` extension. The SPIN tool (`spin`) takes a Promela file and can either: simulate an execution of the model it describes; or do an exhaustive search of that model to check that desirable properties hold true. In either case, a run of SPIN with a Promela file results in a report being generated on `stdout`. In the event that a verification check fails, it also outputs a counterexample in the form of a "trail file". These are not very informative, as they are a list of number triples. However, SPIN can then be invoked to take a trail file and re-simulate its execution, with a report also sent to `stdout`.

SPIN takes many command-line arguments and is very versatile. It takes a Promela model and generates a C program that is tailored (and optimised) to do a simulation or verification of precisely that model. The C program is usually produced in a file called `pan.c` (and `pan.h`), with a number of support files of the form `pan.*`.

2.2.1 Promela/SPIN

SPIN is lightweight, written C, requiring only `gcc`, `lex` and `yacc` to build. It can be obtained from http://spinroot.com/. Note that is already part of current stable Debian releases (`/usr/bin/spin`), and Ubuntu 16.10+.

2.3 Generating Tests from Promela/SPIN

The first main task was exploring how to automatically generate tests from Promela/SPIN, as has been done by others [UAHB10] .

We explored two approaches, based on a common idea:

1. Construct a Promela model of the code to be tested.
2. Add `printf` statements to generate output that was easy to recognise and parse.
3. Provide a text-based mapping from the `printf` output to relevant C-code fragments.
4. Provide a tool that builds the corresponding C test code.

The two approaches differed in how Step 2 above was handled, and also influenced the way Step 3 was implemented.

> Manual: the `printf` statements are added to the model by hand.
>
> Automatic: the `printf` statements are added to the model by a tool following some heuristics.

The two approaches are complemetary to some degree, and where initially supported by seperate tools. These tools have been merged to some degree.

CHAPTER THREE

TOOLS

3.1 Tools Introduction

All the tools provided are written in either *Python3*, or a python language extension package called *Coconut* that adds in basic pattern-matching. We make use of Python virtual environments in the same way as is done at the top-level for the QDP tools.

At present, there are three command-line tools available:

spin2test

testgen

testbuilder

3.2 Spin2Test

The *spin2test* tool is designed to take a single counterexample from a Promela model written using the manual annotation approach along with some support files (mostly C and Header file fragments), and use them to generate RTEMS test code ready for deployment. A key support file is a YAML dictionary that maps.

3.3 TestGen

This tool now merges the functionality of *spin2test* with functionality that supports the automatic annotation approach. It takes a sequence of Promela files as input. Some of these files contain just Promela comments, within which are embedded commmands for the tools, and refinement mappings from annotations to C code.

It relies on modified versions of third party python code including a Promela parser and code to handle C comments. See Appendix 10.2 for details.

3.4 TestBuilder

The *testbuilder* tool takes a number of commands from the command line that help to automate various phases of test generation. It invokes SPIN to generate all counterexamples, and can then be used to systematically generate test code for all of them, by invoking *spin2test* each time. It can also be used to deploy the tests to the appropriate location in RTEMS, to build them, and run them.

CHAPTER FOUR

USER MANUAL

4.1 User Manual Introduction

Here provide all the details needed to setup and use the test generation tools.

4.2 System Pre-Requisites

To build the tools and use them to generate tests requires:

`python3`

`pip3`

Promela/SPIN, from http://spinroot.com

To deploy and run the testcode requires an installation of both `modules/rtems` and `modules/rsb` from the ESTEC *rtems-smp-qualification-qual* repository. The tools should be installed for RTEMS 6, following the instructions in:

https://docs.rtems.org/branches/master/user/index.html

In the examples below we assume a build for a SPARC architecture such as `leon3`, `gr712rc`, or `gr740`. In principle these tests can be built and run on any architecture.

4.3 Building Tools

The test generation tools are found in *formal/promela/src* .

To build, enter that directory and issue the commands:

```
make env
. env/bin/activate
```

To avoid using (long) absolute pathnames to the tools directory, it is worth defining some environment variables and aliases:

```
SPIN2TEST="/..../formal/promela/src/manual"
export SPIN2TEST
alias spin2test="${SPIN2TEST}/spin2test.sh"
alias testgen="${SPIN2TEST}/testgen.sh"
alias tbuild='python3 $SPIN2TEST/testbuilder.py'
```

4.4 Running tools

4.4.1 Spin2Test

The *spin2test* tool is run from a directory containing a Promela model along with supporting files, all linked together by a common model name

For a model called *mymodel* we expect to see the following files:

- *mymodel.pml* - the Promela model
- *mymodel-rfn.yml* - the model refinement to C test code
- *tc-mymodel.c* - the testcase setup C file
- *tr-mymodel.h* - the test-runner header file
- *tr-mymodel.c* - the test-runner setup C file

The following files are templates used to assemble a single test-runner test file for each scenario generated by the Promela model:

- *mymodel-pre.h* - preamble material at the start
- *mymodel-run.h* - test runner material
- *mymodel-post.h* - postamble material at the end.

See the Event Manager (Section 6) for examples.

4.4.1.1 Steps to use *spin2test* to generate tests

To generate all the test scenarios, invoke SPIN as follows:

```
spin -run -E -c0 -e mymodel.pml
```

This will generate N trail files, numbered 1..N:

mymodel.pml1.trail . . . mymodel.pmlN.trail

Annotation Generation

Use SPIN to produce listings with the @@@ annotations for I in {1,..,N} as follows:

```
spin -T -t1 mymodel.pml > mymodel-0.spn
spin -T -t2 mymodel.pml > mymodel-1.spn
...
spin -T -tN mymodel.pml > mymodel-{N-1}.spn
```

SPIN numbers files from 1 up, whereas RTEMS prefer to number things, including filenames, from zero.

Test Code Generation

Run *spin2test* on each *spn* file as follows:

```
spin2test mymodel 0
spin2test mymodel 1
....
spin2test mymodel {N-1}
```

These will generate test-runner test files as follows:

```
tr-mymodel-0.c
tr-mymodel-1.c
...
tr-mymodel-{N-1}.c
```

Test Code Deployment

All files starting with *tc-* or *tr-* need to be copied to the relevant testsuite directory. At present, this is *testsuites/validation* at the top level in the *rtems* repository. All the names of the above files with a *.c* extension need to be added into the testsuite specification directory, into a YAML file that defines the Promela generated-test sources. At present, this is *spec/build/testsuites/validation/model-0.yml* at the top-level in the *rtems* repository.

They should appear under the *source* key something like this:

```
source:
- testsuites/validation/ts-model-0.c
...
- testsuites/validation/tr-model-0.c
- testsuites/validation/tr-model-1.c
...
- testsuites/validation/tr-mymodel-{N-1}.c
...
```

Building the Test Executable

Just navigate to *rtems* and give the command *./waf*

The test executable will be stored in a build directory.

At present this is *build/sparc/gr712rc/testsuites/validation/ts-model-0.exe*, assuming that *sparc/gr712rc* appears in *config.ini*.

Running the Test Executable

Use the SIS simulator:

```
sparc-rtems6-sis -leon3 -r s -m 2 <path-to>/ts-model-0.exe
```

In case of *sparc/gr740* specified in *config.ini*, *-leon3* should be replaced by *-gr740* and can be used with *-m 4* to run with 4 cores.

4.4.2 TestBuilder

The *testbuilder* tool is also run from a directory configured as for *spin2test* above. Basically it automates most of the steps described above.

It requires a file *testbuilder.yml* that supplies key pathnames so it knows where to find relevant tools and where to put the generated tests. The file *formal/promela/src/testbuilder-template.yml* is a template that should be edited to match where RTEMS and RSB are installed, and saved as *formal/promela/src/testbuilder.yml*

If we have setup *tbuild* as an alias to run *tbuilder*, then the following sequence of commands will generate, deploy, and run tests as described above for *spin2test*:

```
tbuild generate mymodel
tbuild copy mymodel
tbuild compile
tbuild run
```

Other commands include *tbuild help* and *tbuild clean*.

A planned future upgrade is to add a single command *tbuild all* that does a *clean* followed by the four steps above.

4.4.3 TestGen

See Appendix 10.2.

4.5 Manual Annotations

We use *printf* statements in the Promela models to output information that is used by *spin2test* to generate test code. This information is used to lookup keys in a YAML file that defines the mapping to C code.

4.5.1 Annotation Syntax

Format, where $N \geq 0$:

`@@@ <pid> <KEYWORD> <thing1> ... <thingN>` where $N \geq 0$

Things:

<pid> Promela Process Id of proctype generating annotation

<word> chunk of text containing no white space

<name> Promela variable/structure/constant identifier

<type> Promela type identifier

<tid> OS Task/Thread/Process Id alias

_ unused argument (within containers)

KEYWORDS and their "things":

```
LOG <word1> ... <wordN>
NAME <name>
INIT
DEF <name> <value>
DECL <type> <name> [<value>]
DCLARRAY <type> <name> <value>
TASK <name>
SIGNAL <name> <value>
WAIT   <name> <value>
STATE tid <name>
SCALAR (<name>|_) [<index>] <value>
PTR <name> <value>
STRUCT <name>
SEQ <name>
END <name>
CALL <name> <value1> ... <valueN>
```

4.5.2 Annotation Refinement

The way that code is generated depends on the keyword in the annotation. In particular, the keyword determines how, or if, the YAML refinement file is looked up.

lookup Mechanisms:

Direct Output - no lookup (LOG, DEF)

Keyword Refinement - lookup (the) <KEYWORD> (NAME, INIT, SIGNAL, WAIT)

Name Refinement - lookup (first) name (TASK, DECL, DCLARRAY, PTR, CALL, SCALAR)

The same name may appear in different contexts, and so we extend the name with a suffix of the form '_XXX' to lookup less frequent uses:

_DCL - A variable declaration

_PTR - The pointer value itself

_FSCALAR - A scalar that is a struct field

_FPTR - A pointer that is a struct field

Generally, the keyword, or name is used to lookup *mymodel-rfn.yml* to get a string result. This string typically has substitution placeholders, as used by the Python *format* method for strings. The other "things" in the annotation are then substituted in, to produce a segment of test code.

Consider the following annotation, from the Events Manager model:

```
@@@ 1 CALL event_send 1 2 10 sendrc
```

This uses Name refinement so we lookup `send` and get back the following text:

```
T_log( T_NORMAL, "Calling Send(%d,%d)", mapid( ctx, {1}), {2} );
{3} = ( *ctx->send )( mapid( ctx, {1} ), {2} );
T_log( T_NORMAL, "Returned 0x%x from Send", {3} );
```

We then substitute in arguments 1 2 10 and sendrc to get the code:

```
T_log( T_NORMAL, "Calling Send(%d,%d)", mapid( ctx, 2), 10 );
sendrc = ( *ctx->send )( mapid( ctx, 2 ), 10 );
T_log( T_NORMAL, "Returned 0x%x from Send", sendrc );
```

This is all implemented in the source file:

formal/promela/src/src/refine_command.coco

4.6 Automatic Annotation Modelling

This is still fairly experimental, and relevant material is in the appendix at

CHAPTER

FIVE

CASE STUDY 1: THE CHAINS API

The first case study was exploring how to automatically generate tests from Promela/SPIN models. This was initially done by looking at a part of the Chains API, with a focus on getting end-to-end results, from a Promela model, to real test code, actually run on real hardware.

5.1 Chain API Models ("Pure")

We now use excerpts from our (sequential) Chain API models to explain the basics of the manual approach. The fragments shown below are based on this early prototype phase, but have been updated to reflect the concurrency enhancements introduced for the Event Manager work.

See https://docs.rtems.org/branches/master/c-user/chains.html

It is quite a large API, with most of it in use somewhere in RTEMS code. For this initial Promela/SPIN study we focussed on just two API calls (ignoring the protected/unprotected distinction for now):

1. Chain Append

   https://docs.rtems.org/branches/master/c-user/chains.html#rtems-chain-append

2. Chain Get

   https://docs.rtems.org/branches/master/c-user/chains.html#rtems-chain-get

We used a model of a system, that performed a mixture of "appends" and "gets", to develop the test-generation idea. The resulting test has been successfully run using `sis`, and on the Remote Test Executor provided by ESTEC. Since then, we have generated more tests using the most recent version of our software, and these have been delivered as part of

Our focus here is on the process of going from a Promela model to a test, so we shall just look at a simpler example that just involves `rtems-chain-append`. As Promela does not have pointers, we re-coded the append algorithm using arrays with pointers being array indices. We treat array index 0 as the equivalent of a NULL pointer, so the first array element is never used.

```
typedef Node { unsigned nxt : PTR_SIZE; unsigned prv : PTR_SIZE; byte itm}

Node memory[MEM_SIZE] ;
```

(continues on next page)

(continued from previous page)

```
typedef Control {
  unsigned head : PTR_SIZE; unsigned tail : PTR_SIZE; unsigned size : PTR_SIZE
}

Control chain ;
```

Promela syntax is based on that of C, with some differences which we will highlight when we encounter them below.

The chains implementation is a doubly-linked list of "nodes" that are accessed from a special "control" structure, using some subtle union overlays to ensure that node access can be done uniformly (no NULL pointer in any node). We abstract considerably from these details for now. In particular, we added an explicit `size` component to the Promela *model* we are developing, to allow us to easily write properties about chain length, and to prevent array out-of-bound errors in the model itself.

We then produced a faithful array version of the append code:

```
inline append(ch,np) {
  assert(np!=0); assert(ch.size < (MEM_SIZE-1));
  if :: (ch.head == 0) -> ch.head = np; ch.tail = np; ch.size = 1;
                          memory[np].nxt = 0; memory[np].prv = 0;
     :: (ch.head != 0) -> memory[ch.tail].nxt = np; memory[np].prv = ch.tail;
                          ch.tail = np; ch.size = ch.size + 1;
  fi
}
```

Here `inline` is a Promela keyword that behaves much like the `#define` of C. Its only difference is that compile-/run-time errors will report line numbers in the *inline* definition, rather than within an expansion where it is used. Promela files can also contain *#define* and SPIN involves *cpp* as part of its processing.

We then create a Promela process that simply calls `append`, and then terminates.

```
proctype doAppend(int addr; int val) {
  atomic{ memory[addr].itm = val; append(chain,addr); } ;
}
```

The keyword *proctype* introduces process code that will run in parallel with other processes. The *atomic* keyword ensures that any run of its contents will not interelave with runs of other processes. Here we are using this to model the protected form of append.

We then run instances of this process in parallel (using the `run` keyword), each inserting different nodes from the array into the chain. This gives us six different interleavings of those append calls.

```
run doAppend(6,21); run doAppend(3,22); run doAppend(4,23);
```

In Promela, each `run` executes its argument in its own "thread". So, in the above, the three calls to `doAppend` run together, interleaving as much as is possible. When SPIN is used in verification

or checking mode, it will explore **all** possible interleavings. In simulation mode, it will show just one, usually chosen at random.

The outcome of the above will a chain of size 3 containing one each of values 21, 22 and 23 in some order. In particular, we expect that `chain.size` would equal 3 at the end. We can indeed get SPIN to verify this by adding an assertion at the end.

```
assert (chain.size == 3);
```

However, this "pure" model of `rtems_chain_append()` is not, of itself, useful for test generation. It needs some adjusting, which is the topic of the next section.

5.2 Annotated Models

We want to use the SPIN model-checker to generate tests. But, like most model-checkers, it only generates output showing some form of execution sequence when it finds a property violation. The model described above always satisfies all its `assert` statements.

So, we trick the model-checker, by negating some assertion that we know to be true (because we model-checked it).

```
assert (chain.size != 3);
```

Now, when we try to verify this model again, what happens is that the model-checker finds this assertion to be false when it reaches it, so it prints out a counter-example. This includes a sequence of steps through the model that leads to this "failure". In fact, this is now an example of a complete correct behaviour of the modelled system.

5.3 Manual Annotation Approach

Here, the modeller adds `printf` statements to the Promela model to make key "features" easy to identify. Then it is necessary to define a mapping from the key features to corresponding C test code. This mapping is known as a "refinement relation". It links abstract concepts from a specification or model to concrete notions in an implementation language. When a counter-example is generated, the resulting output file is parsed to collect a sequence of features, which are then translated using the refinement mapping into a sequence of code snippets. These are then merged with some C template files to produce a complete RTEMS test program.

The addition of suitable feature `printf` statements into a Promela file has to be done manually here, as does the definition of the refinement mapping. However, the generation of counter-examples, their feature extraction, and translation to C code is all automated.

5.3.1 Manual Annotations

The syntax we adopted for feature annotations was to use lines of the form

```
@@@ pid KEY parameters
```

Every annotation line starts with a marker (@@@) that is easy to identify. This is followed by `pid`, a number that is the identifier, of the Promela process printing the annotation. This not required for the sequential chain model, but is important for the concurrent models. This is followed by `KEY`, a keyword describing the kind of feature, and then we supply zero or more (string) parameters. The parameters have no "meaning" as such, and are viewed as strings to be copied as-is or mapped to some refinement string, before being output to the test program.

Our main interest is making the calls to the append API visible, along with the results of such a call. We do this by adding in two statements to the `doAppend` process

```
proctype doAppend(int addr; int val) {
  atomic{ memory[addr].itm = val; append(chain,addr);
          printf("@@@ 0 CALL append %d %d\n",val,addr); show_chain(); } ;
}
```

The `printf` statement output indicates a call (`CALL`) to the append API with the actual values supplied for parameters `addr` and `val`. The statement `show_chain()` is an inline function that prints the contents of the resulting chain after append returns. The resulting output is multi-line, starting with `@@@ 0 SEQ chain`, ending with `@@@ 0 END chain`, and with entries in between of the form `@@@ 0 SCALAR _ val` displaying chain elements, line by line.

We need more than just API calls annotated in this way. We also have to provide annotations for various declarations. These have to appear in the Promela main program (called `init`) as they have to be printed out as the model starts to run. The atomic initialiser becomes somewhat larger:

```
atomic{
  printf("\n\n Chain Model running.\n");
  printf("\n\n Chain Model running.\n");
  printf("@@@ 0 NAME Chain_AutoGen\n")
  printf("@@@ 0 DEF MAX_SIZE 8\n");
  printf("@@@ 0 DCLARRAY Node memory MAX_SIZE\n");
  printf("@@@ 0 DECL unsigned nptr NULL\n")
  printf("@@@ 0 DECL Control chain\n");

  printf("\nInitialising...\n")
  printf("@@@INIT\n");
  chain.head = 0; chain.tail = 0; chain.size = 0;
  show_chain();
} ;
```

The problem is that a `#define`, or a type or variable declaration, is a compile-time feature of the Promela language, so it won't output useful information at runtime. Here we are adding `printf` statments to the `init` block in Promela model to output this information. **Note:** This is

a good candidate for useful annotations that might be generated by the automatic annotation system described in the next section.

Note that we show the initialised chain at the end.

We can now run the Promela model using SPIN in verification mode, to generate a "counter-example". This is done in two steps: the first writes the counter-example to a "trail file"; while the second replays this trail file to run the counter-example.

```
spin -run chains.pml
spin -t -T chains.pml > chains.spn
```

It is also possible to ask SPIN to search for all possible counter-examples, which, in this case, would produce a number of trail files. There would be at least one trail file for each possible run of the system.

One possible result of running SPIN to get counter-example output is shown below. When we filter it to keep just the lines starting with `@@@` we get:

```
@@@ 0 NAME Chain_AutoGen
@@@ 0 DEF MAX_SIZE 8
@@@ 0 DCLARRAY Node memory MAX_SIZE
@@@ 0 DECL unsigned nptr NULL
@@@ 0 DECL Control chain
@@@ 0 INIT
@@@ 0 SEQ chain
@@@ 0 END chain
@@@ 0 PTR nptr 0
@@@ 0 CALL append 22 3
@@@ 0 SEQ chain
@@@ 0 SCALAR _ 22
@@@ 0 END chain
@@@ 0 CALL append 21 6
@@@ 0 SEQ chain
@@@ 0 SCALAR _ 22
@@@ 0 SCALAR _ 21
@@@ 0 END chain
@@@ 0 CALL append 23 4
@@@ 0 SEQ chain
@@@ 0 SCALAR _ 22
@@@ 0 SCALAR _ 21
@@@ 0 SCALAR _ 23
@@@ 0 END chain
```

We have a section (`NAME` ... `DECL`) that does declarations specific to this test, and we use `INIT` to indicate where test initialization has to be done. The first `SEQ` ... `END` pair is intended to display the initial chain, which should be empty. The rest of the output is a series of sequences starting with `CALL` that show the arguments given to `rtems_chain_append`, and then followed by `SEQ`, `SCALAR`, ..., `END` showing the resulting chain contents.

The `NAME` line must be the first line, and should not occur elsewhere.

5.3.2 Manual Refinement

In order to create a test program, we need to map (a.k.a. "refine") the symbolic execution trace into concrete C code. At the top-level, involves building a C test program from some hand-written boiler plate C code, combined with the generated C code. The hand-written code will typically include a preamble and a postamble.

This refinement mapping is defined two ways. The first is in a YAML file that describes a Python dictionary that maps a name to some C code, with placeholders that are used to allow for substituting in actual test values. The second is by writing C code that can help in producing test objects. This C code can be part of the hand-written preamble.

The YAML file has entries of the following form:

```
entity: |
  C code line1{0}
  ...
  C code lineM{2}
```

The "entity" is a reference to the name of some variable or type in the Promela model. There can be zero or more arguments in the annotations. and any occurrence of braces enclosing a number in the C code means that the corresponding annotation argument string is substituted in (using the python string object *format* method).

We don't present a full description of the refinement mapping format here, but rather just explain the features used in this small example. A full description can be found in the User Manual section (4.

`DEF MAX_SIZE 8` This does not require looking up the mapping. We imply replace `DEF` by `#define` and return `#define MAX_SIZE 8`.

`DCLARRAY Node memory MAX_SIZE` This is mapping a Promela array variable to the corresponding C array base type. Lookup the name Promela variable name `memory_DCL` in the mapping:

```
memory_DCL: item {0}[{1}];
```

We generate the C code: `item memory[MAX_SIZE];`.

`DECL Control chain` Lookup the name `chain_DCL` in the mapping and get back result `rtems_chain_control`, and generate C code `rtems_chain_control chain;`.

`INIT` This marks the point where C initialisation code is required. Lookup `INIT` in the mapping:

```
INIT: rtems_chain_initialize_empty( &chain );
```

This returns C code `rtems_chain_initialize_empty( &chain );`.

This could be a multi-line outcome giving a sequence of initialisation steps, or it could call a general initialisation function defined elsewhere.

`CALL append 23 4` Here we map a concrete API call in Promela to the C equivalent. Lookup `append` in the mapping, supplying arguments *23* and *4*:

```
append: |
  memory[{1}].val = {0};
  rtems_chain_append( &chain, (rtems_chain_node*)&memory[{1}] );
```

We substitute `{0}` and `{1}` by *23* and *4* to obtain:

```
memory[4].val = 23;
rtems_chain_append( &chain, (rtems_chain_node*)&memory[4] );
```

SEQ chain

We need a way to handle aggregate data like lists or arrays. The `SEQ` keyword triggers a search for zero or more occurrences of `SCALAR _ sval`, accumulating a string composed of the `sval` arguments separated by spaces, until an `END` is found.

Here we have a situation where how we translate a name like `chain` depends on the context in which it appears. We use different suffixes starting with an underscore to distinguish these usages.

So, *chain_SEQ* is looked up in the mapping with its `{0}` bound to the accumulated `sval` string:

```
chain_SEQ: |
  show_chain( &chain, ctx->buffer );
  T_eq_str( ctx->buffer, "{0} 0" );
```

For the first instance of `SEQ chain`, the accumulated value of the `sval` is `""`, while for the last instance it is `"23 22 21"`. In the latter case we generate the following C code:

```
show_chain( &chain, buffer );
T_eq_str( buffer, "23 22 21 0" );
```

Function `show_chain()` is defined in the C preamble, and displays the chain as a string that is formatted in exactly the same way as per the translation. This is an example of a piece of hand-written C code used to define the overall refinement mapping.

The refinement YAML file contains a number of other translation features we don't discuss here, designed to deal with generating tests for pointers and C structs, and handling concurrency features. More details about these can be found in the User Manual section of this document (Section 4).

CHAPTER

SIX

CASE STUDY 2: THE EVENT MANAGER

We then looked at working on the Event Manager, in parallel with work on it being done in Task 2. This is more complex, due to the presence of concurrent tasks.

The Event Manager is a central piece of code in RTEMS SMP, being at the basis of task communication and synchronization. It is used for instance in the implementation of semaphores or various essential high-level data-structures, and used in the Scheduling process. At the same time, its implementation is making use of concurrent features of C11, and contains many unprotected interactions with the Threads API. Having a Promela model faithfully modelling the Event Manager code of RTEMS represent thus a real challenge, especially with respect to formal testing. This application constitutes as well a way to measure the completeness of our manual and automatic test generation tools previously developed.

The RTEMS Event Manager was chosen as the second case-study because it involved concurrency and communication, had a small number of API calls (just two), but also had somewhat complex requirements related to task priorities.

The Event Manager allows tasks to send "events" to, and receive "events" from, other tasks. From the perspective of the Event Manager, "events" are just uninterpreted numbers in the range 0..31, encoded as a 32-bit bitset.

`rtems_event_send(id,event_in)` allows a task to send a bitset to a designated task

`rtems_event_receive(event_in,option_set,ticks,event_out)` allows a task to specify a desired bitset with options on what to do if it is not present.

Most of the requirements are pretty straightforward, but two were a little more complex, and drove the more complex parts of the modelling.

1. If a task was blocked waiting to receive events, and a lower priority task then sent the events that would wake that blocked task, then the sending task would be immediately preempted by the receiver task.
2. There was a requirement that explicitly discussed the situation where the two tasks involved were running on different processors.

6.1 Event Manager Model Overview

The Event Manager model (`event-mgr-model.pml`) consists of five Promela processes.

`init` is the first top-level Promela process it performs initialisation, starts the other processes, waits for them to terminate, and finishes.

`System` is a Promela process that models the behaviour of the operating system, in particular that of the scheduler.

`Clock` is a Promela process used to facilitate modelling timeouts

`Sender` is a Promela process used to model the RTEMS sender task.

`Receiver` is a Promela process used to model the RTEMS receiver task.

The interaction between the `System` and the task processes is facilitated by modelling RTEMS task states (Ready, Blocked, etc.) in a somewhat ad-hoc manner.

6.2 Annotating Concurrency

In the Chain API model, we used concurrent Promela processes to invoke chain operations with fairly arbitrary interleavings, and then used a given interleaving as a test scenario, implemented as a single-threaded program that did the API calls in the order given by the interleaving. So if we wanted to produce an annotation giving the value of variable `var` at a given point in time, and given its value happens to be 42, we would produce the following output:

```
@@@SCALAR var 42
```

From this we generate test code to check that `var` has value 42.

In the Events Manager, we typically have at least two RTEMS Tasks running in parallel, signalling each other. So it is necessary to track which task is doing what. What this means is that we need to know the Promela process id, so the annotation above, assuming this was process 3, would be:

```
@@@ 3 SCALAR var 42
```

We also need annotations to provide information about the relationship between Promela process identifiers, and the RTEMS Tasks we propose to use to do the testing. In effect this is were we are in fact coding some refinement information in the Promela annotations. To say that Promela process 3 corresponds to RTEMS test program Task 0, we would use the following annotation:

```
@@@ 3 TASK 0 3
```

These annotation can be written in the Promela model without the developer having to figure out the process identifier. This is because Promela has a special variable `_pid` that returns the process id (similar to `self` for Tasks). The following Promela code generates the above two annotations, assuming the process they are in has process id 3 and represent what is intended to be Task 0:

```
printf("@@@ %d TASK %d %d\n",_pid,0,_pid);
printf("@@ %d SCALAR var %d",_pid,var);
```

Given a sequence of the above kind of annotations, as a result of a particular counterexample interleaving, we also gain some very important information. We need to devise a test which reproduces that same interleaving.

The information regarding when tasks should wait and/or restart can be obtained by tracking the process identifiers, and noting when they change. The `spin2test` program does this, and also produces separate test code segments for each Promela process.

So, given the file `model-events-mgr-0.spn`, for instance, we see annotation from Promela process 0, 3 and 4, so the generated C code in `tr-model-events-mgr-0.c` has the following exceprt (with some lines elided):

```
/* ===== TEST CODE SEGMENT 0 =====*/
static void TestSegment0( Context* ctx ) {
  /* Test Name is defined in the Test Case code (tc-model-events-mgr.c) */
  T_log(T_NORMAL,"@@@ 0 INIT");
  initialise_pending( pending, TASK_MAX );
  initialise_semaphore( ctx, semaphore );
}

/* ===== TEST CODE SEGMENT 3 =====*/
static void TestSegment3( Context* ctx ) {
  T_log(T_NORMAL,"@@@ 3 TASK Worker");
  checkTaskIs( ctx->worker_id );
  T_log(T_NORMAL,"@@@ 3 WAIT 0");
  Wait( semaphore[0] );
  ....
  T_log(T_NORMAL,"@@@ 3 STATE 0 Zombie");
  /* Code to check that Task 0 has terminated */
}

/* ===== TEST CODE SEGMENT 4 =====*/
static void TestSegment4( Context* ctx ) {
  T_log(T_NORMAL,"@@@ 4 TASK Runner");
  checkTaskIs( ctx->runner_id );
  ...
  T_log(T_NORMAL,"@@@ 4 STATE 1 Zombie");
  /* Code to check that Task 1 has terminated */
}
```

In order to generate code to manage the synchronisation appropriately, we need to use SIGNAL and WAIT annotations, that can be refined into Task synchronisation code as appropriate.

So for example, the annotation:

```
@@@ 4 WAIT 1
```

has the corresponding refinement entry:

```
WAIT: |
  Wait( semaphore[{}] );
```

which results in the followeding C code:

```
Wait( semaphore[1] );
```

This combination of always having the Promela process identifier in the annotation, along with explicit WAIT and SIGNAL annotations, makes it straightforward (so far) to manage concurrent test Tasks.

CHAPTER

SEVEN

CASE STUDY 3: MRSP THREAD QUEUES

The final case study of Task 3.2 was to use Promela/SPIN and the tools developed so far, to look at the correctness of the Thread Queue Implementation.

7.1 Current State of Play

Below, we summarise the current state of the thread queue verification effort. All this verification material can be found at *formal/promela/models/threadq*.

In subsequent sections we give more detail about what has been done.

7.1.1 Scoping

Scoping is complete, and at first glance seems reasonable in size. We are looking at scenarios involving a number of processors and schedulers running a number of tasks of varying priorities that simply create, obtain, and release MrsP semaphores. In practise, this touches a large part of the RTEMS code base. We need to handle a wide variety of queues, implemented using both chains and red-black trees, as well as different locking protocols. The datastructures that represent processors, schedulers and threads, are complex, with many linkages in between them. The MrsP protocols require task to migrate from one processor to another under certain circumstances.

7.1.2 Modelling

Modelling began by looking at the key RTEMS API calls involved, namely *rtems_semaphore_create()*, *rtems_semaphore_obtain()*, and *rtems_semaphore_release()*. Progress was good until an assertion from the RTEMS source failed when transcribed into the Promela model. This raised the need to model how the entire system is initialized, at least those parts that can influence the MrsP protocol behaviour. RTEMS initialization is very complex, and an initial working model has only just been completed.

7.1.3 Validation

In the current state of the model, the main methods of validation are: careful reading of the Promela code with respect to the corresponding C code; and implementing every C code assertion in Promela using the *assert()* construct. The C assertions capture the implementors understanding of good behaviour, and our model should at least check they are satisfied within the model.

7.1.4 Verification

We can perform simulation runs to observe behaviour, but the model is not at the stage where we can use the model-checker to check high-level properties, such as deadlock- or live-lock freedom.

7.2 Scoping Details

We defined the top-level scope of our work as being the use of MrsP semaphores, in a setting were the usage is deemed to be correct. This means that we do not expect to get any error responses from the API calls, and so these need not be modelled. This is a reasonable assumption given that we are looking at systems that are being proposed to go through ECSS qualification, and we would expect bad API usage to be screened out elsewhere.

Concretely, we assumed the following settings would apply to all the code:

Defined macros:

RTEMS_SMP

RTEMS_SCORE_CPUSTDATOMIC_USE_STDATOMIC

Undefined macros:

RTEMS_DEBUG

RTEMS_PROFILING

RTEMS_MULTIPROCESSING

Semaphore usage attributes, that define the MrsP protocol:

RTEMS_PRIORITY

RTEMS_BINARY_SEMAPHORE

RTEMS_MULTIPROCESSOR_RESOURCE_SHARING

Almost all of the relevant source code is found in *modules/rtems/cpukit*, except for the red-black three implementation code that resides in the cross-compilation tool-suite, at *sparc-rtems6/include/tree.h*.

From the semaphore API starting points, following the code visits a lot of the RTEMS source tree. From *cpukit* we looked in:

- *include/rtems* - 24 header source files (*.h*)

- ***rtems* - 7 code source files, including the semaphore API calls,** in *semcreate.c*, *semobtain.c* and *semrelease.c*.
- *score* - 1 header and 9 code source files.

We also looked at *modules/rtems/testsuites/smptests* for existing test programs aimed at SMP and MrsP behaviour.

Local copies of these can be found in `formal/promela/models/threadq/srccpy/rtems.

7.3 Modelling Details

All the Promela models can be found under *formal/promela/models/threadq/*.

Initial modelling addressed low-level issues, such as C11 atomics, and weak memory models, and is found in the *Weak-Memory* folder. They won't be discussed here.

7.3.1 SPARC TSO Model

A formal model has been built of the SPARC Total Store Order (TSO) memory model, as a complete standalone model as *Weak-Memory/SPARC-TSO.pml*. This models the write-buffer associated with each CPU, and the way that reads by a CPU after a write to the same address can get the value from that buffer.

7.3.2 Configuration Generator Test

A model was developed to explore the use of Promela's non-deterministic choice construct as a way to generate a wide range of scenarios.

We set some upper limits for the numbers of processor cores (4), schedulers (4), tasks per core (2), with a maximum of 6 tasks. We also assumed at most 4 resources being protected by semaphores.

We then defined a number of constraints that relate the above components:

- each core is managed by one scheduler
- a scheduler can manage multiple cores
- a task is associated with one scheduler
- a scheduler handles multiple tasks
- a task has a priority
- a task uses a number of resources
- a resource is used by a number of tasks
- a resource has a ceiling priority for every scheduler which is at least as high as the highest priority of any task on that scheduler that uses that resource.

We then defined a non-deterministic process to setup a scenario, as follows:

1. choose number of cores - core_count > 0
2. choose number of schedulers - sched_count <= core_count
3. allocate cores to all available schedulers
4. choose number of tasks - task_count >= core_count
5. choose number of resources - at least one.
6. Allocate resources to tasks, at least one resource per task

In addition we had facilities to generate fixed size scenarios, so we could

Experimentation with this model showed that the "configuration space" is very large. It is easy to run Spin in simulation model to obtain a random configuration. Attempting to get Spin to do a full model verification is much more difficult.

We did some dummy test generation runs, to see how many Spin trail files we observed. Even if we fixed the numbers of cores, schedulers, tasks, and resources, we found that the number of paths through the system was very high.

Fixing all those values to 2, and allowing 4 different priority values in the model, resulting in 192 trail files. If we allow a 5th priority level, the trail file count rises to 1300. If we keep the priority levels to 4, but all 3 cores, schedulers and tasks, the number rises to 2304. If we allow 3 resources as well, we obtain 25088 trail files. It is very clear that we need to be very careful with scenario generation.

The model is defined in the file *ConfigTest.pml* and is a standalone model.

7.3.3 MrsP Code Model

Models found in *formal/promela/models/threadq/MrsP-Code*.

We want to be able to model the behaviour of a number of RTEMS tasks trying to access resources protected by a number of MrsP semaphores, in a general multi-core setting. We want the model to be able to cover all the ways in which the tasks can interleave and access as they follow the relevant MrsP protocols in order to obtain, and then release, the relevant semaphores.

There are quite a few aspects of the overall concurrent behaviour that need to be modelled:

- Concurrent interleaving
- Scheduler context switching
- Processor affinity
- Algorithmic details
- Atomicity
- C pointer usage

The Promela language makes use of the C pre-processor, and we make use of this facility to help structure the models. The model mainline resides in *MAIN.pml*, and it, and other PML files, use the #include directive to bring in definitions from elsewhere.

7.3.3.1 Datatypes

There are a large number of RTEMS datatypes involved. Most of these are modelled in the file *Structs.pml*.

A particular issue arises from the fact that some structs contain nested versions of other structs.

For example:

- *Thread_queue_Context* has an internal struct *Path* that has a field *Links* that is a *Chain_Control* structure.
- *Thread_queue_Link* has a field *Path_node* that is a *Chain_Node* structure.
- Thread Scheduler control has an embedded *Chain_Node* and *two* embedded *Chain_Controls*.

These nesting can be modelled with Promela *typedefs*, but the C code often takes the *address* of these internal structures, for example when allowing an embedded *Chain_Node* to be inserted into a chain. We cannot replicate this directly in Promela.

All C pointers have to be converted into array indices. Any embedded struct has to be extracted and placed in its own array, with the struct field converted to an array index. This considerably raises the complexity, and lowers the maintainability, of these models.

Another complication is that the `inline` construct in Promela that we use is basically a text substition construct, so we cannot declare local variables within them. This means we have to be careful with our name-spaces.

The following naming convention is proposed to assist in the management of this:

- For an embedded struct called *SubStruct* whose address is used, we replace it with an array index called *iSubStruct*.
- For a C pointer called *pointer*, we replace it with *pointeri*.
- For a local variable *var*, we replace it with *XXvar1[taskno]*, where *XX* is a short string that identifies the relevant function. If the same name is used in a lower call that can have a different value, then we increment the number (*XXvar2[..]*,`XXvar3[]`,..).
- **One exception is for initialization,** where there is only one Promela process active, where we use the form *XX_var*.
- Another exception is where an *inline* has an *atomic* body, in which case there will be no interference by another process.

7.3.3.2 Queues

The use of red-black trees for priority queues, and the RTEMS Chains API for FIFO queues, is modelled in files *RBTrees.pml* and *Chains.pml* respectively. We model their behaviour using a simulation, with arrays and array indices, of a singly-linked list implementation.

The key aspects of concurrent behaviour, such as interleaving, context-switching, processor affinity, are modelled in *Concurrency.pml*. This allows us to put markers in our model code to indicate where a thread of execution might block, or get pre-empted, and when it would also

resume. It also defines a property that ensures our model never claims to have more than one execution thread running on any processor at any one time.

7.3.3.3 Initialization

The initialization of MrsP, and by extension, of much of the CPU, scheduler and task datastructures has been modelled in Promela, in *Init.pml*. This has been the most complex part to date.

RTEMS initialization is a mix of static structures setup by the sources under *cpukit/include/rtems/confdefs*, referenced via *cpukit/include/rtems/condefs.h*, and dynamic initialization code that is accessed via function pointers from those statically initialized components. Unravelling this has taken a lot of time and involved visiting a lot of code: at one point going deep enough to encounter linker-related code! A key aspect has been factoring out the relevant parts of the initialization.

The current state of *Init.pml* is that it appears to be complete. However, certain priority nodes associated with tasks and their schedulers still seem to be unitialised (value zero, the default value in Promela).

This really needs expert input to provide a picture of what a correctly initialized RTEMS SMP system looks like. In particular, how are all the *Priority_Nodes* setup before any semaphore is created, and how they all look as each semaphore comes into existence.

7.3.3.4 Semaphore Operations

The three key semaphore API calls (create,obtain,release) are in the process of being defined in *Semaphores.pml*.

The *rtems_semaphore_create()* operation is modelled so we can ensure that such a semaphore is initialized correctly. We only cater for MrsP semaphores, and we assume that all usage of them is correct, in that we always use valid semaphore ids, and always have an agreed order in which semaphores are obtains and released.

The modelling of the *rtems_semaphore_obtain()* is underway, but stalled at a point where we started to model the process by which a task's priority gets raised to that of the semaphore it is trying to access. This is the entry point to the most complex part of the MrsP protocol. This is a function called *_MRSP_Raise_priority*, and should succeed with a success .return code We use a Promela *assert()* in our model to check this is so. The relevant snippet from our model is the following, from *Semaphores.pml*:

```
inline _MRSP_Claim_ownership( aseqno, // action sequence number
                              mrsp_id, // more actual parameters
                              rc ) // return value
{
  printf("_MRSP_Claim_ownership(...)\n");

  printf(" MrsP Prio = %d\n",
          PN[sema_info[mrsp_id].MRSP.Ceiling_priority]._priority);
  printf(" Before: Thread Prio = %d\n",
```

(continues on next page)

(continued from previous page)

```
        PN[_TControl[aseqno].Real_priority]._priority);
  _MRSP_Raise_priority( aseqno,
                        mrsp_id,
                        sema_info[mrsp_id].MRSP.Ceiling_priority,
                        rc );
  printf(" After: Thread Prio = %d\n",
        PN[_TControl[aseqno].Real_priority]._priority);
  assert( rc == STATUS_SUCCESSFUL ); // We expect success here
  printf("_Claim NYfII\n");
}
```

The *assert()* on the second last line fails. The SPIN output just before this point is as follows:

```
_MRSP_Claim_ownership(...)
MrsP Prio = 2
Before: Thread Prio = 0
MRSP Raise priority(...)
aseqno=1, mrsp_id=0, priority_node=2
MRSP Raise priority: acquired ticket lock
_TControl[aseqno].Scheduler.home_scheduler = 1
RPschedNode[aseqno] = 2
ceiling_priority is 2
ceil_prio = 2
PN[]._priority = 2
NOT-OK: ceil_prio > ShedNode priority
MRSP Raise priority: released ticket lock
After: Thread Prio = 0
spin: ./Semaphores.pml:266, Error: assertion violated
spin: text of failed assertion: assert((status==RTEMS_SUCCESSFUL))
```

The thread priority should be 2.

7.3.4 MrsP Abstract Model

Models found in *formal/promela/models/threadq/MrsP-Test*.

This abstract model is intended to model the behaviour of MrsP semaphores at fairly high level, in order to be able to generate tests. By a fairly high level we mean that key MrsP protocol events such as "helping" are not visible, nor is the consequent task migration. What we expect to see are tasks having their priorities raised to the ceiling priority of a semaphore when it is being, and has been, obtained. We expect to observe a FIFO queue for semaphores, and priorities dropping once a semaphore has been released. We also want to ensure that a Ready task is only Running under a given scheduler when there is no other Ready task with a higher priority under that same scheduler. In effect, the PIP protocol running as if the extra cores didn't complicate matters.

The real point of this high level model is to make it possible to generate a wide range of testable scenarios that will exercise all the corner and boundary cases of the underlying implementation.

The top-level of the model is in file *mrsp-threadq-model.pml*. It calls inline code from included files (*Sizing*, *Configure*, *Run*) to initialise and run the model. It waits for all tasks to terminate, and then, if environment variable `TEST_GEN` is defined, it runs an *assert(false)* statement, to trigger counterexample generation.

All the Promela files use printf statements to report decisions made and behaviours generated.

A scenario is characterised by three key numbers or cores, tasks, and resources (each with its own semaphore). We require at least one task per core, and at least two tasks.

Sizing is about making a non-deterministic selection of those sizes, and also provides some datastructures to record the resulting choices.

Currently the model is setup to handle a maximum of 4 cores, 6 tasks, and 4 resources.

(see *Sizing.pml*)

Given the result of sizing, we then need to generate a configuration by non-deterministically associating tasks with both cores and resources. We need to ensure that every core has at least one task, and every resource is used by at least two tasks.

Even with the contraints above, there are a large number of combinations. The configuration code tries to exploit symmettries in the problem to reduce this number. For this, we make a key assumption, that in the model, no core or task gets preferential treatment in any way. In effect, every task becomes Ready at the same time on its given core, and all subsequent behaviours are decided by the same scheduler and semaphore rules applied impartially.

So, for example, given C cores and T tasks ($C \leq T$), we assign the first C tasks in task number order to the cores, also in the same core number order. We can do this "without loss of generality" (w.l.o.g.).

Configuration first associates tasks with cores, and then resources with tasks. Once these (non-deterministic) choices are made, then the ceiling priorities, per core, for each resource's semaphore, are pre-determined and we calculate and record those.

(see *Configure.pml*)

Once configuration is complete, then we start to run the dynamic behaviour. This includes the following components:

- FIFO queues, implemented as circular buffers in Promela.
- Task states (NotInUse, Ready, Running, Blocked) and transitions (START,DISPATCH,PREEMPT,BLOCK,UNBLOCK,TERMINATE), along with a model of how transitions occur (*contextSwitch*).
- Semaphore behaviour, for both obtain and release, with management of task priority changes and FIFO queue updates.
- Task behaviour, which is basically to obtain semaphores for its resources in a standard system-wide order, then release them in the reverse order, and terminate.

The top-level simply sets up all tasks in the Ready state, each as a Promela process, and lets them rip.

The approach in the Event manager model to deal with context switching was ad-hoc. In this thread-queue model it is more more systematic, as evidenced by the following code snippet modelling a running task:

```
proctype Task( byte tno ) {
  byte freed;

  printf("@@@ %d TASK %d\n", _pid, tno);
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryObtain( tno, 0 );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryObtain( tno, 1 );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryObtain( tno, 2 );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryObtain( tno, 3 );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  printf("Task %d has all its resources\n",tno);
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryRelease( tno, 3, freed );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryRelease( tno, 2, freed );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryRelease( tno, 1, freed );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  tryRelease( tno, 0, freed );
  contextSwitch( taskConfig[tno].taskCore );
  WAIT_TO_RUN( tno );
  runState[tno] = NotInUse // TERMINATE
  printf("@@@ %d LOG Task %d prio:%d cpu:%d -TERMINATE-> %e\n", _pid,
         tno, runPrio[tno], taskConfig[tno].taskCore, runState[tno]);
  printf("@@@ %d STATE %d %e\n", _pid, tno, runState[tno]);
  contextSwitch( taskConfig[tno].taskCore );
}
```

(see *Run.pml*)

SPIN can be asked to generate all possible paths through the model with the following command-line invocation:

```
spin -DTEST_GEN -run -E -c0 -e mrsp-threadq-model.pml
```

In a model with 3 cores, 3 tasks, and 2 resources, and exhaustive check produces 1092 scenarios, each as a SPIN trail file. The full output for scenario 618 is in the appendix at 10.1 .

It reports the following setup:

```
System Setup

  Task Cores, Priorities, and Resources
    Task 0 on core 0 with priority 1
      Resources used: 0 1
    Task 1 on core 1 with priority 5
      Resources used: 0 1
    Task 2 on core 2 with priority 3
      Resources used: 1

  Resource Tasks, and Core Priorities
    Resource 0 used by 2 task(s)
     Tasks making use: 0 1
     Core ceiling priorities (core@prio): (0@1) (1@5) (2@0)
    Resource 1 used by 3 task(s)
     Tasks making use: 0 1 2
     Core ceiling priorities (core@prio): (0@1) (1@5) (2@3)
```

Here is an extract of an initial part of the scenario output that will drive test generation:

```
@@@ 0 DEF core_count 3
@@@ 0 DEF task_count 3
@@@ 0 DEF res_count 2
@@@ 0 LOG Task 0, prio:1 cpu:0 -START-> Ready
@@@ 0 STATE 0 Ready
@@@ 0 LOG Task 0 prio:1 cpu:0 -DISPATCH-> Running
@@@ 0 STATE 0 Running
@@@ 1 TASK 0
@@@ 1 CALL obtain 0 0
@@@ 1 LOG semaphore 0 was free
@@@ 1 CALL setTaskPriority 0 1 1
@@@ 1 TASK 0 USING SEMA 0 is 1
@@@ 0 LOG Task 1, prio:5 cpu:1 -START-> Ready
@@@ 0 STATE 1 Ready
@@@ 0 LOG Task 1 prio:5 cpu:1 -DISPATCH-> Running
@@@ 0 STATE 1 Running
@@@ 2 TASK 1
@@@ 0 LOG Task 2, prio:3 cpu:2 -START-> Ready
@@@ 0 STATE 2 Ready
@@@ 0 LOG Task 2 prio:3 cpu:2 -DISPATCH-> Running
@@@ 0 STATE 2 Running
@@@ 3 TASK 2
@@@ 3 CALL obtain 2 1
@@@ 3 LOG semaphore 1 was free
@@@ 3 CALL setTaskPriority 2 3 3
@@@ 3 TASK 2 USING SEMA 1 is 1
...
```

7.4 Validation Details

At present, the main form of validation is ensuring that assertions in the RTEMS code get reflected as Promela *assert* statements.

What has emerged as a key issue is a need to have a specification of how scheduler thread queues, and similar, are initialized. This should describe the outcome in high-level terms.

7.5 Verification Details

We have nothing to report here at present.

CHAPTER

EIGHT

CASE STUDY 4: THE BARRIER MANAGER

The Barrier Manager was developed by a Masters student, Jerzy Jaśkuć [Jaskuc22], using the Event Manager model as a reference.

8.1 Manager Overview

The documentation of the Barrier Manager can be found in the RTEMS Classic API Guide at https://docs.rtems.org/branches/master/c-user/barrier/index.html .

It provides a facility, or "barrier", to force a set of tasks to synchronise by calling a service that makes them wait until the whole set has invoked that service. Then they are all free to proceed.

There are five directives defined. Three are involved with creating, naming and removing barriers, while two deal with waiting at, and releasing, these barriers.

Barriers work in one of two different modes:

- In *automatic* mode a barrier has an associated limit number. Tasks wait as long as the number waiting is less than that limit. Once the number of tasks at the barrier reaches the limit, then they are all released.
- In *manual* mode, there is no limit on the number of waiting tasks. They are only released when some other non-waiting task explicity releases them.

8.2 Model Overview

All five directives are modelled.

The top-level model is similar to that done for the Events Manager. There are two processes, *System* and *Clock*, that model relevant RTEMS infrastructure, such as the scheduler, and timing with timeouts.

In addition there are three processes that model RTEMS Tasks, one called the *Runner*, along with two workers *Worker0* and *Worker1*. Three binary semaphores are used to coordinate task running. All three processes set their own priority, and choose their own processor core if the scenario requires multicore. Then the workers block on their own binary semaphore, while the *Runner* process then creates barriers. Once done, it releases the worker binary semaphores, and then may attempt to wait at, release, or delete the barriers. It then waits for the runners to complete

Both workers have the same behaviour, initially being blocked until the *Runner* has set things up. They too attempt to wait at, release, or delete the barriers.

8.3 Scenarios Overview

There are three top-level scenarios: *ManAcqRel*, *AutoAcq*, *AutoToutDel*, and each process can participate independently in these. The runner process always does the barrier creation.

The *ManAcqRel* scenario deals with creating, acquiring and releasing barriers in *manual* mode. Within this it has sub-scenarios that deal with error cases (invalid name, null pointers).

The *AutoAcq* scenario deals with creating, acquiring and releasing barriers in *automatic* mode. The main error case handled is one with too many processes waiting on a barrier.

The *AutoToutDel* scenario also runs is *automatic* mode and looks at timeout handling and barrier deletion.

All the above three scenarios can also run in a single-core or a multicore setting.

8.4 Comparison with Event Manager

A considerable amount of the model is for underlying infrastructure that is essentially the same as that provided in the Event model. This includes the following inlines: *outputDefines*, *output-Declarations*, *Obtain*, *Release*, *waitUntilReady*, *preemptIfRequired*.

The *satisfied* inline is different as expected, here identifying when a waiting process has been released.

A key difference between RTEMS Events and RTEMS Barriers is that event sets are always associated with the receiving Task (their “pending” set), while barriers have an identity independent of any given task.

CHAPTER

NINE

CASE STUDY 5: THE MESSAGE MANAGER

The Message Manager was developed by a Masters student, Eoin Lynch [Lyn22], using the Event Manager model as a reference.

9.1 Manager Overview

The documentation of the Message Manager can be found in the RTEMS Classic API Guide at https://docs.rtems.org/branches/master/c-user/message/index.html

It provides a message queue facility that allows Tasks to communicate with each other. A message queue is a dynamic entity whose size can change, with user-defined contents.

There are ten directives defined. Four are involved with creating, naming and removing message queues, while six deal with different ways to read, write and modify queues. There is also a mechanism to define a message queue buffer.

Message queues have two configurable attributes:

- A *queueing mode* mode for taks that are waiting to receive a message. This can be either FIFO or based on priority
- A *locality* mode in that a queue can be either local or global.

9.2 Model Overview

Only four directives are modelled. These are two ways to construct a queue (*message_queue_construct*, *message_queue_create*), along with the basic send and receive calls (*message_queue_send*, *message_queue_receive*).

The top-level model is similar to that done for the Events Manager. There are two processes, *System* and *Clock*, that model relevant RTEMS infrastructure, such as the scheduler, and timing with timeouts.

In addition there are three processes that model RTEMS Tasks, one called the *Sender*, along with two receivers *Receiver0* and *Receiver1*. Three binary semaphores are used to coordinate task running.

The *Sender* creates message queues, wakes up one of the receivers, and then sends a variable number of messages. It then waits for the receivers to complete.

The first receiver *Receiver0* performs a message receive operation and then either wakes up the sender or other receiver depending on the scenario.

The second receiver *Receiver1* only runs in some scenarios. It performs a receive, and then wakes up the sender process.

9.3 Scenarios Overview

There are four top-level scenarios (*Send*, *Receive*, *SndRcv*, *RcvSnd*), which are similar in concept to the Event Manager scenarios.

The *Send* scenario just has one process sending, with no receivers. This is mainly used to test out errors when calling *message_queue_send()*.`

The *Receive* scenario just has one process receivimg, with no senders. This is mainly used to test out timeout options and errors when calling *message_queue_receive()*.

The *SndRcv* scenario has both sending and receiving processes running. Here the sender starts first and sends 1 or 2 messages and the receivers are configured to wait for messages.

The *RcvSnd* scenario has a sending and one receiving processes running. Here the receiver starts first, with a timeout set. The sender then sends a message.

9.4 Comparison with Event Manager

A considerable amount of the model is for underlying infrastructure that is essentially the same as that provided in the Event model. This includes the following inlines: *outputDefines*, *output-Declarations*, *Obtain*, *Release*, *Released*, *waitUntilReady*.

A key difference between RTEMS Events and RTEMS Messages is that event sets are always associated with the receiving Task (their “pending” set), while message queues have an identity independent of any given task.

CHAPTER TEN

APPENDIX

This appendix contains the following large reference items:

- The full output of MrsP Scenario 618/1092 (10.1).
- The full description of the *testgen* tool (10.2).
- How to automatically extract Promela models from C11 code (10.3).

10.1 MrsP Scenario 618

This is the output generated for scenario 618 (out of 1092) from the asbtract MrsP model (*formal/promela/models/threadq/MrsP-Test/*):

```
THREADQ-MAIN capacity: 4 cores, 6 tasks, 4 resources

Fixing Counts
  Core count:     3
  Task count:     3
  Resource count: 2
@@@ 0 DEF core_count 3
@@@ 0 DEF task_count 3
@@@ 0 DEF res_count 2

Generating deployment...

Setup Priorities
  Assigning Priorities to Tasks
    Task 0 on core 0 has priority 1
taskIx = 1
    Task 1 on core 1 has priority 5
taskIx = 2
    Task 2 on core 2 has priority 3
taskIx = 3
  Computing Ceiling Priorities

System Setup
```

(continues on next page)

(continued from previous page)

```
  Task Cores, Priorities, and Resources
    Task 0 on core 0 with priority 1
      Resources used: 0 1
    Task 1 on core 1 with priority 5
      Resources used: 0 1
    Task 2 on core 2 with priority 3
      Resources used: 1

  Resource Tasks, and Core Priorities
    Resource 0 used by 2 task(s)
     Tasks making use: 0 1
     Core ceiling priorities (core@prio): (0@1) (1@5) (2@0)
    Resource 1 used by 3 task(s)
     Tasks making use: 0 1 2
     Core ceiling priorities (core@prio): (0@1) (1@5) (2@3)

Run - task_count = 3

Running Tasks....

@@@ 0 LOG Task 0, prio:1 cpu:0 -START-> Ready
@@@ 0 STATE 0 Ready
@@@ 0 LOG Task 0 prio:1 cpu:0 -DISPATCH-> Running
@@@ 0 STATE 0 Running
@@@ 1 TASK 0
@@@ 1 CALL obtain 0 0
@@@ 1 LOG semaphore 0 was free
@@@ 1 CALL setTaskPriority 0 1 1
@@@ 1 TASK 0 USING SEMA 0 is 1
@@@ 0 LOG Task 1, prio:5 cpu:1 -START-> Ready
@@@ 0 STATE 1 Ready
@@@ 0 LOG Task 1 prio:5 cpu:1 -DISPATCH-> Running
@@@ 0 STATE 1 Running
@@@ 2 TASK 1
@@@ 0 LOG Task 2, prio:3 cpu:2 -START-> Ready
@@@ 0 STATE 2 Ready
@@@ 0 LOG Task 2 prio:3 cpu:2 -DISPATCH-> Running
@@@ 0 STATE 2 Running
@@@ 3 TASK 2
@@@ 3 CALL obtain 2 1
@@@ 3 LOG semaphore 1 was free
@@@ 3 CALL setTaskPriority 2 3 3
@@@ 3 TASK 2 USING SEMA 1 is 1
Task 2 has all its resources
@@@ 1 CALL obtain 0 1
@@@ 1 LOG Task 0 prio:1 cpu:0 -BLOCK-> Blocked
@@@ 1 STATE 0 Blocked
```

(continues on next page)

(continued from previous page)

```
@@@ 1 LOG semaphore 1 in use
@@@ 1 CALL setTaskPriority 0 1 1
@@@ 1 TASK 0 USING SEMA 1 is 1
@@@ 3 CALL release 2 1
@@@ 3 LOG Task 0 prio:1 cpu:0 -UNBLOCK-> Ready
@@@ 3 STATE 0 Ready
@@@ 3 LOG semaphore 1 still in use
@@@ 3 CALL setTaskPriority 0 1 1
@@@ 3 LOG Task 0 prio:1 cpu:0 -DISPATCH-> Running
@@@ 3 STATE 0 Running
@@@ 3 CALL setTaskPriority 2 3 3
@@@ 3 LOG Task 2 prio:3 cpu:2 -TERMINATE-> NotInUse
@@@ 3 STATE 2 NotInUse
Task 0 has all its resources
@@@ 1 CALL release 0 1
@@@ 1 LOG semaphore 1 was freed
@@@ 1 CALL setTaskPriority 0 1 1
@@@ 1 CALL release 0 0
@@@ 1 LOG semaphore 0 was freed
@@@ 1 CALL setTaskPriority 0 1 1
@@@ 2 CALL obtain 1 0
@@@ 2 LOG semaphore 0 was free
@@@ 2 CALL setTaskPriority 1 5 5
@@@ 2 TASK 1 USING SEMA 0 is 1
@@@ 2 CALL obtain 1 1
@@@ 2 LOG semaphore 1 was free
@@@ 2 CALL setTaskPriority 1 5 5
@@@ 2 TASK 1 USING SEMA 1 is 1
Task 1 has all its resources
@@@ 2 CALL release 1 1
@@@ 2 LOG semaphore 1 was freed
@@@ 2 CALL setTaskPriority 1 5 5
@@@ 2 CALL release 1 0
@@@ 2 LOG semaphore 0 was freed
@@@ 2 CALL setTaskPriority 1 5 5
@@@ 2 LOG Task 1 prio:5 cpu:1 -TERMINATE-> NotInUse
@@@ 2 STATE 1 NotInUse
@@@ 1 LOG Task 0 prio:1 cpu:0 -TERMINATE-> NotInUse
@@@ 1 STATE 0 NotInUse
.. all tasks launched
All tasks have terminated
spin: trail ends after 1476 steps
#processes: 1
              taskConfig[0].taskCore = 0
              taskConfig[0].taskPrio = 1
              taskConfig[0].taskRes[0] = 1
              taskConfig[0].taskRes[1] = 1
```

(continues on next page)

(continued from previous page)

```
taskConfig[0].taskRes[2] = 0
taskConfig[0].taskRes[3] = 0
taskConfig[1].taskCore = 1
taskConfig[1].taskPrio = 5
taskConfig[1].taskRes[0] = 1
taskConfig[1].taskRes[1] = 1
taskConfig[1].taskRes[2] = 0
taskConfig[1].taskRes[3] = 0
taskConfig[2].taskCore = 2
taskConfig[2].taskPrio = 3
taskConfig[2].taskRes[0] = 0
taskConfig[2].taskRes[1] = 1
taskConfig[2].taskRes[2] = 0
taskConfig[2].taskRes[3] = 0
taskConfig[3].taskCore = 0
taskConfig[3].taskPrio = 0
taskConfig[3].taskRes[0] = 0
taskConfig[3].taskRes[1] = 0
taskConfig[3].taskRes[2] = 0
taskConfig[3].taskRes[3] = 0
taskConfig[4].taskCore = 0
taskConfig[4].taskPrio = 0
taskConfig[4].taskRes[0] = 0
taskConfig[4].taskRes[1] = 0
taskConfig[4].taskRes[2] = 0
taskConfig[4].taskRes[3] = 0
taskConfig[5].taskCore = 0
taskConfig[5].taskPrio = 0
taskConfig[5].taskRes[0] = 0
taskConfig[5].taskRes[1] = 0
taskConfig[5].taskRes[2] = 0
taskConfig[5].taskRes[3] = 0
resSetup[0].resTaskCount = 2
resSetup[0].resTasks[0] = 1
resSetup[0].resTasks[1] = 1
resSetup[0].resTasks[2] = 0
resSetup[0].resTasks[3] = 0
resSetup[0].resTasks[4] = 0
resSetup[0].resTasks[5] = 0
resSetup[0].resPrio[0] = 1
resSetup[0].resPrio[1] = 5
resSetup[0].resPrio[2] = 0
resSetup[0].resPrio[3] = 0
resSetup[1].resTaskCount = 3
resSetup[1].resTasks[0] = 1
resSetup[1].resTasks[1] = 1
resSetup[1].resTasks[2] = 1
```

(continues on next page)

(continued from previous page)

```
resSetup[1].resTasks[3] = 0
resSetup[1].resTasks[4] = 0
resSetup[1].resTasks[5] = 0
resSetup[1].resPrio[0] = 1
resSetup[1].resPrio[1] = 5
resSetup[1].resPrio[2] = 3
resSetup[1].resPrio[3] = 0
resSetup[2].resTaskCount = 0
resSetup[2].resTasks[0] = 0
resSetup[2].resTasks[1] = 0
resSetup[2].resTasks[2] = 0
resSetup[2].resTasks[3] = 0
resSetup[2].resTasks[4] = 0
resSetup[2].resTasks[5] = 0
resSetup[2].resPrio[0] = 0
resSetup[2].resPrio[1] = 0
resSetup[2].resPrio[2] = 0
resSetup[2].resPrio[3] = 0
resSetup[3].resTaskCount = 0
resSetup[3].resTasks[0] = 0
resSetup[3].resTasks[1] = 0
resSetup[3].resTasks[2] = 0
resSetup[3].resTasks[3] = 0
resSetup[3].resTasks[4] = 0
resSetup[3].resTasks[5] = 0
resSetup[3].resPrio[0] = 0
resSetup[3].resPrio[1] = 0
resSetup[3].resPrio[2] = 0
resSetup[3].resPrio[3] = 0
core_count = 3
task_count = 3
res_count = 2
taskIx = 3
coreIx = 3
remTasks = 0
possibleP = 0
chosenP = 0
coresLeft = 0
partSize = 0
prevSize = 0
smallestP = 0
largestP = 0
resIx = 2
resTCount = 0
available = 0
target = 0
tskResCount = 1
```

(continues on next page)

(continued from previous page)

```
trIx = 0
prioUsed[0] = 1
prioUsed[1] = 1
prioUsed[2] = 0
prioUsed[3] = 1
prioUsed[4] = 0
prioUsed[5] = 1
prioUsed[6] = 0
prioUsed[7] = 0
prioUsed[8] = 0
prioUsed[9] = 0
prioUsed[10] = 0
prioUsed[11] = 0
prioUsed[12] = 0
prioUsed[13] = 0
prioUsed[14] = 0
prioUsed[15] = 1
core = 2
prio = 3
resprio = 0
q = 0
runState[0] = NotInUse
runState[1] = NotInUse
runState[2] = NotInUse
runState[3] = 0
runState[4] = 0
runState[5] = 0
coreHiPrio[0] = 0
coreHiPrio[1] = 0
coreHiPrio[2] = 0
coreHiPrio[3] = 0
tix = 0
semaphores[0].inuse = 0
semaphores[0].owner = 1
semaphores[0].q.buf[0] = 0
semaphores[0].q.buf[1] = 0
semaphores[0].q.buf[2] = 0
semaphores[0].q.buf[3] = 0
semaphores[0].q.buf[4] = 0
semaphores[0].q.buf[5] = 0
semaphores[0].q.head = 0
semaphores[0].q.tail = 0
semaphores[1].inuse = 0
semaphores[1].owner = 1
semaphores[1].q.buf[0] = 0
semaphores[1].q.buf[1] = 0
semaphores[1].q.buf[2] = 0
```

(continues on next page)

(continued from previous page)

```
                semaphores[1].q.buf[3] = 0
                semaphores[1].q.buf[4] = 0
                semaphores[1].q.buf[5] = 0
                semaphores[1].q.head = 1
                semaphores[1].q.tail = 1
                semaphores[2].inuse = 0
                semaphores[2].owner = 0
                semaphores[2].q.buf[0] = 0
                semaphores[2].q.buf[1] = 0
                semaphores[2].q.buf[2] = 0
                semaphores[2].q.buf[3] = 0
                semaphores[2].q.buf[4] = 0
                semaphores[2].q.buf[5] = 0
                semaphores[2].q.head = 0
                semaphores[2].q.tail = 0
                semaphores[3].inuse = 0
                semaphores[3].owner = 0
                semaphores[3].q.buf[0] = 0
                semaphores[3].q.buf[1] = 0
                semaphores[3].q.buf[2] = 0
                semaphores[3].q.buf[3] = 0
                semaphores[3].q.buf[4] = 0
                semaphores[3].q.buf[5] = 0
                semaphores[3].q.head = 0
                semaphores[3].q.tail = 0
                runPrio[0] = 1
                runPrio[1] = 5
                runPrio[2] = 3
                runPrio[3] = 0
                runPrio[4] = 0
                runPrio[5] = 0
                waitingFor[0] = 1
                waitingFor[1] = 0
                waitingFor[2] = 0
                waitingFor[3] = 0
                waitingFor[4] = 0
                waitingFor[5] = 0
                runi = 3
                fix = 1
1476: proc  0 (:init::1) mrsp-threadq-model.pml:79 (state 439) <valid end state>
4 processes created
```

10.2 Description of *testgen*

This documentation provides an informal specification of the tool (*testgen*) used to generate RTEMS-SMP test cases, from Promela models. It covers external aspects such as how the tool should be configured and invoked. It also describes internal aspects, like the syntax and semantics of the annotation language designed for use with Promela, as well as the notation used to define the mapping from Promela model concepts to RTEMS C test code.

The document also describes the current nature and state of its implementation.

10.2.1 Specification of *testgen*

We have to specify the various modes in which the toolset can be used. Then we have to specify two "languages": one used for model annotations, and another used to refine model concepts to code. We also need to specify how documentation for the tool should be produced. Finally we note external constraints on the implementation, largely determined by RTEMS community considerations.

10.2.1.1 Tool Usage

Patterns of use; Invocation

The test code produced by `testgen` should be deployed within RTEMS as part of the "standard" test setup. This means that these tests will be run and reported just like any other RTEMS tests.

10.2.1.2 Promela Annotations

Used to highlight key model events and behaviours

10.2.1.3 Refinement

Linking Promela concepts to correspond ideas in RTEMS C code. We note that our approach is (intentionally) target language agnostic.

10.2.1.4 Documentation

The RTEMS qualification toolset under development contains enhancements to documentation generation that allow much to be generated automatically. Of particular note are the new specification items, designed so that both test-code, and documentation can be generated from one source file.

We should adopt a similar approach to the documentation of `testgen`, in so for as this is practicable.

10.2.1.5 Constraints

In keeping with both ESA and RTEMS Community guidelines, the implementation language for this generator is Python3. An effort should be made to minimise the number of extra Python packages brought on board. The toolset should be part of the RTEMS qualification tools, and be installed using *make env*, like all other Python tools.

10.2.2 Implementation of *testgen*

This section details how to invoke our test generation tool to produce test files from Promela model.

The technique is generic: it has been successfully applied to produce C test files, but also "meta" YAML test files (that can themselves be used at a later stage to generate more concrete programming-language specific code, including C).

The information making our tool producing specific language implementation code is captured in various additional data that we provide to our tool as input. Refinement files are examples of such information needed by the tool to run.

The next sections start by giving a basic description of how folders are organized, and go deeper to present the semantics of input format and refinement files of our test generation tool.

10.2.2.1 Structure of the Source

Examples

Running examples for the tool are located in `FV2-201_models`, whereas `Promela_to_C/examples` is only relevant for configuring the current model-checker invoked by the tool. Examples in `FV2-201_models` are mostly independent, and thus can be tried out separately.

By taking `FV2-201_models/chains` as example, abbreviated as `models_chains`, we list here its content:

- `models_chains/rqmts-spec-level/freechain/cpukit/1_pure/cpukit.pml`: Minimal Promela model without propositions and side-effects (e.g., without assert and printf).
- `models_chains/rqmts-spec-level/freechain/cpukit/2_printf/cpukit.pml`: Variant based on `models_chains/rqmts-spec-level/freechain/cpukit/1_pure/cpukit.pml` — Example of printing with printf to incrementally show the states of various data.
- `models_chains/rqmts-spec-level/freechain/testsuites_spfreechain01.pml`: Variant based on `models_chains/rqmts-spec-level/freechain/cpukit/1_pure/cpukit.pml` where the refinement part and configuration are separate:
  - `models_chains/rqmts-spec-level/freechain/testsuites_spfreechain01.rfn` — Usage of more elaborated formulae (i.e., ltl proposition)
  - `models_chains/rqmts-spec-level/freechain/testsuites.cfg`
- `models_chains/rqmts-spec-level/freechain/testsuites/testsuites_spfreechain01.pml`: Variant based on `models_chains/rqmts-spec-level/freechain/testsuites_spfreechain01.pml` — Example of assertion using assert.

Besides the example in `models_chains/rqmts-spec-level`, another "low-level" variant is provided in `models_chains/src`, which has actually been specifically derived from `models_chains/examples/only_generic`, and particularly `models_chains/examples/only_generic/chains.pml` and `models_chains/examples/only_generic/chains.rfn`. Like `models_chains/rqmts-spec-level`, we retrieve a similar structure:

- `models_chains/src/testsuites_spfreechain01.pml`
- `models_chains/src/refine_sptests/testsuites_spfreechain01.rfn`
- `models_chains/src/testsuites.cfg`

The folder `models_chains/rqmts-spec-level/errors` regroups several examples of models which specifications might lead to miscellaneous corner cases.

- For instance, the mutual interaction of add and del implemented in `models_chains/rqmts-spec-level/errors/queue_too_many_processes.pml` can lead to an infinite execution loop (see the documentation in the file).
- `models_chains/rqmts-spec-level/errors/queue_invalid_end_state.pml` is an example where the program is terminating in some not expected ending point.

10.2.2.2 Semantics of the Generation

`Promela_to_C/src/testgen_ml.sh` takes an ordered list of Promela *contents* as argument, where a "content" is more specifically defined as being:

- either a *path* of a file containing a Promela source,
- or a raw *string* representing a Promela source, which is directly provided in the command line as argument.

By default, all arguments of `Promela_to_C/src/testgen_ml.sh` are understood to be paths of files. The explicit way to specify that an incoming argument is not a path but a string is by using the option `--promela` (or — equivalently — the shorter `-p` option).

There are basically only two constraints to fulfill for a list of provided arguments be characterized as well-parsed, and subsequently processed with the intended normal behavior of `Promela_to_C/src/testgen_ml.sh`:

1. Each argument must be separately well-parsed by the parser `promela-yacc`, a parser library that we have derived from the initial project https://github.com/johnyf/promela to support more elements of the Promela language, and that we provide in `Promela_to_C/src/src/modules/promela_yacc`.
2. Additionally, the final (ordered) concatenation of all arguments must produce a content well-parsed by the parser integrated in the model-checker M, which value is set at runtime (more details below).

Note that as corollary, the combination of the above constraints makes any prefix concatenation subsets of arguments be well-parsed by `promela-yacc`, as well as well-parsed by the parser of M.

Also, the acceptance range of `promela-yacc` has been modified to additionally accept empty programs as well-parsed entities. This makes a sole invocation of `Promela_to_C/src/testgen_ml.`

sh (without arguments) not failing: in the tiniest setting, `Promela_to_C/src/testgen_ml.sh` first start by compiling the source as usual.

As we mentioned that `Promela_to_C/src/testgen_ml_spin.sh` is a shortcut for `Promela_to_C/src/testgen_ml.sh`, an example to invoke the test generation program would then be:

```
Promela_to_C/src/testgen_ml_spin.sh
    $STRIP/testsuites_spfreechain01.pml
    $STRIP/testsuites_spfreechain01.rfn
    --promela "$(cat $STRIP/testsuites.cfg )"
```

where `$STRIP` is a shorthand for the folder `models_chains/rqmts-spec-level/freechain` or `models_chains/src`.

The motivation and details of the different files are now provided in the following part.

10.2.2.3 Extending Promela with an Annotation Language

Since the Promela (specification) language can be approximated as a strict subset of the C (programming) language, we must necessarily increase the expressivity of the language that our tool is accepting in input, for the task of generating *C* test files from Promela models be first decidable, and second as automatic as possible.

Consequently, not only will our tool take as input some Promela model, but it will also take some additional information specifying how the model will have to be extracted to *C* (together with any information that can not be automatically processed by our tool). Such information is basically what *refinement* files are.

One way of providing the refinement information and model to the tool is by using an *annotation language*, embedded inside Promela comments. Generally, annotations have the advantage of acting as transparent information container with little disruption on the source: e.g. similarly as to how one is taking advantage of programming languages supporting arbitrary comments, to embed there "formal" information. Usually, the formal information can refer to any piece of code mostly considered critical, enabling at the same time any forms of formal reasoning, through a special system of reference quotations and dereferences to the code. However we will see that this is not mandatory, e.g. the formal information can independently coexist with the code and be placed before or next to it, also without making any particular referencing to the code.

A Promela *comment* is similar as a comment written in C: in Promela, there is a preprocessing phase generating Promela tokens from some raw unpreprocessed source. It turns out that the preprocessing language of Promela is strictly identical to the preprocessing language of C. Consequently, we distinguish two types of comments[1]:

- block comments enclosed between `/*` and `*/`,
- line comments starting with `//`.

Additionally, we noticed that `@@` is not a symbol used in the reference manuals of C and Promela that we inspected, also not used in the C and Promela source we dealt with. So we took the

[1] https://gcc.gnu.org/onlinedocs/gcc-7.5.0/cpp/

liberty of changing the priority preprocessing of that symbol, by extending the language of comments and considering the following item as valid:

- line comments starting with `@@`.

Similarly as to how an *annotated comment* is opened in ACSL[2], we use the specific syntactic keyword `@` to distinguish between a regular comment and a comment which content will be additionally submitted to the type system of our annotation language. Overall, annotated comments are taking the following forms:

- block annotated comments enclosed between `/*@` and `*/`,
- line annotated comments starting with `//@`,
- line annotated comments starting with `@@@`.

Inside an annotated comment, we might want to use consecutive *commands* to better organize the flow of information arising from our interaction with the underlying formal system (i.e. our test generation tool). A command is composed of a *command-identifier* taking several *command-arguments*.

We can thought of commands as ordered lists with possible duplication of elements, where several same command-identifiers can appear in the list. In this setting, we place command-identifiers aligned to the first indentation level, whereas command-arguments are just appearing in the second one.

For example, in the following annotated comment expressed in YAML syntax, $\texttt{cmd}_1$ is taking three arguments, whereas $\texttt{cmd}_2$ four:[3]

```
/*@
    - cmd_1:
        - arg^1_1
        - arg^2_1
        - arg^3_1
    - cmd_2:
        - arg^1_2
        - arg^2_2
        - arg^3_2
        - arg^4_2
*/
```

Equivalently, the same content can be rewritten in a **single** line as follows:[4]

```
//@ [{"cmd_1": ["arg^1_1", "arg^2_1", "arg^3_1"]}, {"cmd_2": ["arg^1_2", "arg^2_2",
"arg^3_2", "arg^4_2"]}]
```

[2] https://en.wikipedia.org/wiki/ANSI/ISO_C_Specification_Language

[3] In some fictive programming language, one could imagine the parsed data to be of type: "((string, string list) dict) list".

[4] Although practically, it might have been rendered split into several lines.

At the time of writing, the above presented YAML syntax is fully supported. There is however an internal syntax currently in effect and called ML. This ML syntax has been more intended for developers, so it might not appear in public releases of the software. As an overview though, we can quickly show how the above annotations are written in ML. First, all command-arguments appear escaped

- between the opening delimiter `\<open>`,
- and the closing delimiter `\<close>`.

For readability reasons, the two enclosing delimiters "`\<open>`" and "`\<close>`" can also be depicted as "‹" and "›".

For example, in the following annotated comment: "`/*@` $\texttt{cmd}_1$ ‹ $\text{arg}^1{}_1$ › ‹ $\text{arg}^2{}_1$ › ‹ $\text{arg}^3{}_1$ › $\texttt{cmd}_2$ ‹ $\text{arg}^1{}_2$ › ‹ $\text{arg}^2{}_2$ › ‹ $\text{arg}^3{}_2$ › ‹ $\text{arg}^4{}_2$ › `*/`", $\texttt{cmd}_1$ is taking three arguments, whereas $\texttt{cmd}_2$ four.

For flexibility reasons, we might want to separate the model information from the annotation information. So our tool accepts to receive in input possibly different types of files containing any information that the user might find relevant to combine together inside a unique file, or leave them explicitly separate.

Generally, even if the list of file arguments is syntactically provided as ordered, it is ultimately the concatenation of the overall content which will be semantically treated by the test generation tool. Since our tool is internally calling the model-checker M, the possibility or not to permute the initial list of arguments that we provide in input will ultimately depend on how well a final concatenation of permuted arguments would be supported by M. In that respect, our tool does not impose a restriction on the particular order of arguments to give as long as what is provided will be eventually correctly treated by M.

From what we mentioned above, the case of "explicit" elements of the language which are semantically treated by M is clear: we are following the same convention as M. On the other hand, "invisible" constructs such as annotation comments are following different freedom rules: they are by default not considered by M and all discarded (by M) in a first preprocessing phase. Consequently, since annotation comments are solely interpreted by our tool, the possibility or not to permute invisible constructs (comments) will differ from the possibility to permute or not more explicit elements of the language.

Besides having flexibility as benefit, with different files the overall maintaining activity can be delegated to different persons: since a Promela file without annotations is standalone, maintaining it only requires a knowledge of Promela. (That would nevertheless require to take care of any informal constructs surrounding any "pure" Promela code, comprising in this case comments, be them formal or not.)

10.2.2.4 Semantics of Commands

Annotation commands are supposed to have the ability to perform "arbitrary" side-effects during their evaluation, e.g. ranging from unrisky behaviour to Turing-complete computation. However, the first group of commands that we designed towards end-users is just a strict subset of annotation commands: being more lightweight, *attribute* commands do not necessitate large computation resources.

Attribute commands take pure information data as arguments: their evaluations will just have the effect of making the information data stored somewhere inside some background context

for later retrieval (for instance, storing them once, persistently, at the start of our generation tool). Later, it will be at the discretion of other non-attribute commands to retrieve any relevant registered data. Registered data are then ultimately understood as a list, so a data registered multiple times will appear multiple times in the list.

Unless otherwise stated, any commands presented in this document belong to the category of attribute commands. To perform the test generation though, there must be obviously other non-attribute commands that our tool is internally using and invoking. However at the time of writing we do not provide an explicit interface for end-users to invoke them. There is one exception though: in the "*C* test generation" part, the user will have the possibility to make the tool execute more powerful pre-defined non-attribute commands, but under some controlled conditions (see the details in that part).

Whereas attribute commands are storing some information data inside a background context, instead of making the data stored "globally", it might be sometimes convenient to store it next to an element *pml_proc_inl* of the Promela model, such as the name of a proctype, the name of an inline element, or the name of any elements which can be uniquely identified. This is to have a handy way to retrieve back the "personalized" data associated to a given unique identifier *pml_proc_inl*. Such retrieval would be mostly beneficial to certain non-attribute commands internally invoked by our tool, while applying algorithms discriminating on the selection of particular node elements of the input model, including the selection order.

To this end, all annotation commands are assumed to be evaluated within a particular *navigation context*: the line where an annotation command is written has now an importance. If a command is written in the same line as an existing Promela proctype or inline declaration, now indifferently abbreviated as *pml_proc_inl*, then that command will be additionally aware that *pml_proc_inl* is available for its use.[5]

However, to simplify our presentation, we will not detail which non-attribute commands are expecting to receive a *pml_proc_inl* as parameter for its correct functioning. Rather, we will sort all attribute commands into two groups, based on whether an attribute command should be placed near a *pml_proc_inl* element, so that its data would be consequently stored, and reused by other commands. This leads to two situations:

- The notation "**(at ‹pml_proc_inl›)**" will indicate that the attribute command should be placed near a valid *pml_proc_inl*. If we can not find a declaration close enough to the command, or if one has provided a name *pml_proc_inl* which does not exist, then the evaluation effect of the command will simply be the identity function.
- The notation "**(globally)**" will be used for attribute commands which will be executed as usual no matter where it was placed.

While developing a non-trivial Promela model of consequent size or for testing reasons, it might be interesting of quickly executing arbitrary actions, involving Turing-complete computation. Such commands will be called *generic* commands. Being the most expressive, they are generalizing de-facto attribute commands.

The high expressivity of generic commands comes nevertheless with a certain price, e.g. regarding their automated execution: our tool will have to execute consecutive generic commands only in sequential order (compared to other types of consecutive "declarative" commands, where more parallel optimization potentials might happen). Reciprocally and still in relation with correctness, providing a set of generic commands, additional cares must be taken while

[5] We assume dealing with at most one proctype or inline per line.

building an enumerative list deciding which command should be the next one to execute from the set: the permutation of two commands might irreversibly produce different side-effects!

Note that some cares must also be brought on the appearance order of certain attribute commands, particularly if a generic command is in the background processing what has been stored by an initial group of attribute commands — more details will be provided in front of any such concerned commands.

With the above warning in mind, at the time of writing, we have only publicly released a subset interface of generic commands (among the many ones internally used by our tool). The precise set of generic commands currently available is detailed in the "Generating a Test File" part.

10.2.3 Generating a Trail File

Providing a Promela file `file.pml`, the role of the model-checker M is to prove that all propositions written in `file.pml` is true. If this is not the case, then there might still be situations where M is able to derive a counter-example in front of a false proposition, showing why the check of `file.pml` failed.

Since the overall operation involves several actions from M, occurring at the time when M is expected to be called, we consequently provide several commands, one for each action.

10.2.3.1 Main Commands

`model_checker_verifier` cts_1 $cts_{n\text{-}1}$ **(globally)** has the same semantics as `model_checker_compile` $cts_1 cts_{n\text{-}1}$ `file.pml`. In particular, `file.pml` is the name of the Promela file that is not needed to explicitly write.

Additionally, all `model_checker_verifier` commands are considered to be ßevaluated before any `model_checker_compile` commands.

`model_checker_compile` cts_1 cts_n **(globally)** declares the shell command "cts_1 cts_n" as command to be executed when M is used to check if all propositions of `file.pml` are true.

Additionally, all `model_checker_compile` commands are considered to be evaluated before any `model_checker_exec_one` commands.

`model_checker_exec_one` cts_1 cts_n **(globally)** declares the shell command "cts_1 cts_n" as command to be executed when M is used to check if all propositions of `file.pml` are true.

Additionally, it is assumed that the last `model_checker_exec_one` cts_1 cts_n shell command is leaving a file named `file.pml.trail` (inside the same directory where it is invoked), in the particular case where M is able to derive a counter-example from `file.pml`. (In case there is no `model_checker_exec_one` command, it is the last `model_checker_compile` or `model_checker_verifier` which is considered in the previous sentence.)

`model_checker_trail` cts_1 $cts_{n\text{-}2}$ **(globally)** declares the shell command "cts_1 $cts_{n\text{-}2}$ `file.pml.trail file.pml`" as command to be executed when M is used to derive a

readable counter-example from `file.pml.trail`. In particular, `file.pml.trail` and `file.pml` are the names that are not needed to explicitly write.

Additionally, it is assumed that the command is directly printing the readable counter-example to the standard output (file descriptor).

10.2.3.2 Optional Commands

`no_printf` (at ‹pml_proc_inl›) declares that *pml_proc_inl* will not have any *C* code to print (irrespective of previous and future use of `refine_uniform`).

`no_atomic` (at ‹pml_proc_inl›) declares that no extra `atomic`^{PML} wrapper will be wrapped around the generated `printf`^{PML} statement, whereas by default, there is one such being generated.

`print_first` (at ‹pml_proc_inl›) declares to insert a `printf`^{PML} at the beginning of the encountered proctype or inline declaration, seen as a sequence of instructions. By default, the insertion is performed at the last position of the sequence.

`disable_negation` (at ‹pml_proc_inl›) declares that no negations of asserts `assert`^{PML} will occur inside *pml_proc_inl*.

10.2.4 Generating a Test File

To generate a *C* test file, we propose two independent approaches: the uniform refinement and the generic refinement. The two approaches can be executed separately or both at the same time, so we could end up with at most two generated *C* files.

10.2.5 _

10.2.5.1 Main Commands

`export_code` *name* (globally) declares to keep the *C* test file inside the location specified by `export_dir` and stored in the file-name *name*.

If this option is not present, the content of all files which were meant to be saved on the file-system are instead discarded (irrespective of the presence of `refine_uniform` or `refine_generic`).

10.2.6 Uniform Refinement

The *uniform refinement* approach gets activated when at least one occurrence of the main commands below is used.

10.2.6.1 Main Commands

`refine_uniform` *c_content* (at ‹pml_proc_inl›) declares the *C* code *c_content* as code to be printed whenever *pml_proc_inl* appears in the counter-example generation.

`refine_uniform_force` (globally) declares that all encountered proctype or inline declaration will receive a generated `printf`^{PML} (encapsulated inside an `atomic`^{PML}, depending on `no_atomic`). By default, a `printf`^{PML} is generated for a *pml_proc_inl* declaration only if there is a `refine_uniform` command associated to that declaration.

The uniform refinement approach inserts several uniform printing markers at key positions in the input model received by `model_checker_verifier`. In return, it extracts from the output of `model_checker_trail` a uniform list L_{cmd} of annotated attribute commands. All attribute commands are uniform in that they are composed of the same command identifier, and only differing in their command arguments. In particular, from these arguments we can retrieve a current *c_content* to print, earlier declared by a corresponding `refine_uniform`. Ultimately, the sequence order of *c_content* is reflecting how elements in L_{cmd} have been arranged. This makes the order of L_{cmd} important, and specially taken in charge by the generation process.

Operationally, it is not *c_content* which will be printed, but a "preprocessed" version of *c_content*: any directives `#refine` appearing in *c_content* will ultimately be rewritten to some equivalent generated *C* code. Other directives are left untouched, so the generated code might again contain "more legal" *C* directives. However if the rewriting of a `#refine` is failing for some reasons, then that particular `#refine` line will be left untouched, and printed as it is in the output.

In more details, `#refine` has the same semantics as `#define` when applied to ground values[6], except that the right hand side (RHS) of the replacement rule is not a static list of *C* tokens, but an interpreted *antiquotation*. This RHS is syntactically of the form:

$$[["ty_{name}", ty_{arity}], "var_{pml}"]$$

where

- var_{pml} is the name of the variable referring to one of the arguments taken by *pml_proc_inl*,
- ty_{name} is the type of var_{pml},
- ty_{arity} is `null` in case var_{pml} is a Promela scalar. Otherwise, it is a positive integer of at most the size of the array ty_{name}. More details on types will be provided below.

Furthermore, some respective lines of `#undef` are appended in the end of *c_content*. For example, if *c_content* is of the form:

$$\texttt{\#refine } x_1 \; [["ty1_{name}", ty1_{arity}], "var_1"]$$

$$\texttt{\#refine } x_2 \; [["ty2_{name}", ty2_{arity}], "var_2"]$$

$$\texttt{\#refine } x_3 \; [["ty3_{name}", ty3_{arity}], "var_3"]$$

`// code`

we will get as output:

[6] The declaration of functional macros is not yet supported by `#refine`.

```
#define x1 value1
#define x2 value2
#define x3 value3
// code
#undef x1
#undef x2
#undef x3
```

assuming that var_1, var_2, and var_3 are all appearing as either global variables or local variables of *pml_proc_inl*. In return, $\texttt{value}_1$, $\texttt{value}_2$ and $\texttt{value}_3$ respectively represent the *serialized* values received by *pml_proc_inl*. These values are automatically computed, serialized and printed by *M* during the counter-example simulation phase.

To serialize values *var*, the tool must know their types. At the time of writing, the type inference process that our tool is performing is rather trivial: it tries to retrieve the types of values by looking at the nearest proctype declaration. If the proctype declaration can not be found, or for parameters of inline declarations or any other global variables, their types *typ* have to be manually provided, using the syntax mentioned above.

If the type of *var* is not ground (e.g. such as the channel type `chan`), some warning or error messages are raised with a default value printed instead for that variable.

The serialization of arrays is supported if we manually provide its size. Note that, for the case of arrays, we might obtain several expanded lines of `#define` and `#undef` behind a single line of `#refine`.

10.2.7 Generic Refinement

The *generic refinement* approach gets activated when at least one occurrence of the main command below is used.

10.2.7.1 Main Command

`refine_generic` *yaml_content* (globally) declares the YAML refinement data *yaml_content* as data to be used as initializer for the computation described below.

The generic refinement approach assumes that one can extract from the output of `model_checker_trail` a list L_{cmd} of annotated generic commands to execute. Each generic command *cmd* in L_{cmd} is then executed one by one. Additionally, each cmd_i is taking as parameter a $state_i$ value, where $state_i$ is the result produced by cmd_{i-1} when applied with $state_{i-1}$ as argument, except that the first element of L_{cmd} is applied with an initializing value computed from *yaml_content*.

Ultimately, we can retrieve from the last computed *state* value some final *C* code to print.

As long as *yaml_content* has been provided once, our tool automatically computes each appropriate *state* value, and ensures that each one gets correctly provided to a corresponding *cmd*.

End-users do not have to manually provide these *state* values. However, there is no automated check performed on the initial list L_{cmd} of generic commands to execute, especially regarding the order of its elements.

At the time of writing, the following generic commands are recognized, the more detailed documentation of the commands and how to invoke them is provided in `FV2-201/src/Promela_to_C/document/TestGeneration.md`:

- *pid* `INIT`
- *cid* `TASK` *tid pid*
- *pid* `NAME` *name*
- *pid* `DEF` *name value*
- *pid* `DECL` *typ name*
- *pid* `DECL` *typ name value*
- *pid* `DCLARRAY` *typ name value*
- *pid* `PTR` *name value*
- *pid* `CALL` *name* [...]
- *pid* `STRUCT` *name*
- *pid* `SEQ` *name*
- *pid* `END` *name*
- *pid* `SCALAR` _ *value*
- *pid* `SCALAR` *name value*
- *pid* `SCALAR` *name value index*
- *pid* `STATE` *tid state*
- *pid* `LOG` [...]

Note that _ denotes a syntactic constant symbol, and [...] represents a variadic list of arguments. Also, all generic commands are acting on the **(globally)** level.

10.2.8 Optional Commands

`enclose` *cts1_header cts2_footer* (globally) declares that *cts1_header* will be printed as header and *cts2_footer* as footer. Multiple `enclose` commands do affect the nested generation order: if we encounter later another `enclose` *cts1_header' cts2_footer'*, then we will ultimately get as printed content: *cts1_header*, *cts1_header'*, , *cts2_footer'*, *cts2_footer*.

`enclose_uniform` *cts1_header cts2_footer* (globally) has the same semantics as `enclose` *cts1_header cts2_footer* except that all contents included by `enclose_uniform` appear nested inside any contents that are ever inserted by `enclose`. Additionally, `enclose_uniform` is only used in the uniform refinement approach.

`refine_uniform_log` (globally) declares that an additional *C* comment will be placed near each *C* function being generated. Each comment contains the name of the *pml_proc_inl* declaration at the origin of the *C* function produced in output.

`refine_uniform_strip` (globally) declares to remove any newlines surrounding *c_content* (in `refine_uniform`).

10.2.9 Generating a Project Directory

10.2.9.1 Main Command

`export_dir` *dir* (globally) declares *dir* as the main directory for any code generated by the tool.

If this option is not present, the content of all files which were meant to be saved on the file-system are instead printed on the standard output.

10.2.9.2 Optional Commands

`export_input_yaml` *dir* (globally) declares to export all command arguments in YAML format within specific annotation comments. Each argument will be exported to a file inside the directory *dir*, which will itself be created inside the directory specified by `export_dir`.

`export_input_only_content` (globally) declares to not include annotation comments while exporting the data specified in `export_input_yaml`.

`export_pml` *name* (globally) declares to keep any intermediate generated promela models inside the location specified by `export_dir` and stored in the file-name *name*.

If this option is not present, the content of all files which were meant to be saved on the file-system are instead discarded.

`export_trail` *name* (globally) declares to keep any intermediate generated output of `model_checker_trail` inside the location specified by `export_dir` and stored in the file-name *name*. It is only the human readable version which is saved (not the default binary file).

If this option is not present, the content of all files which were meant to be saved on the file-system are instead discarded.

10.2.10 Checking Input Format

10.2.10.1 Main Command

`check_unparse_parse` (globally) declares to serialize all (parsed) arguments provided to the program as input, and parse the resulting string. Afterwards, a comparison is made to check that the initial arguments and new ones are equal, and raise errors in case of mismatch.

10.2.11 Derivative Forms

10.2.11.1 Positional Reference

***cmd*_at *arg*$_0$ *arg*$_1$ *arg*$_n$** has the same semantics as "*cmd arg*$_1$ *arg*$_n$ **(at $‹arg›_0$)**", except for commands *cmd* expecting to refer to a *pml_proc_inl* declaration. For these commands, it is no more mandatory that their positions have to be situated in the same line as a corresponding *pml_proc_inl*. Instead, *arg*$_0$ is specially used to indicate which declaration the command is referring to. This makes the writing of the annotation command possible to happen anywhere on a file.

Since we assume for the moment being restricted to the use of Promela models not permitting redefinition of declarations, without loss of flexibility, the definition of an annotation can in this case also occur before the actual proper definition of the declaration.

Non-exhaustive examples of *cmd*`_at` commands include `refine_uniform_at`, or `no_printf_at`.

10.2.11.2 Content as File

***cmd*_file *arg*$_1$ *arg*$_n$** has the same semantics as "*cmd arg'*$_1$ *arg'*$_n$", where *arg'*$_k$ is the whole content of the file stored at path *arg*$_k$.

If the path *arg*$_k$ is not an absolute path, then it is considered to be relative to the input resource from where the command *cmd*`_file` was read before the test generation program started. This leads to two situations:

1. If *cmd*`_file` was read from a file, then the resolving of any relative links of commands inside the file is straightforward, based on the path of that file.
2. If *cmd*`_file` was provided through the shell option `--promela` to the test generation program, then the base directory taken as starting directory for relative path resolving is the directory where the user has executed that program (not necessarily the program's directory).

As a non-exhaustive example of commands particularly using the referencing of content through file, we can cite `enclose_file`.

10.2.11.3 Positional Reference and Content as File

***cmd*_file_at *arg*$_0$ *arg*$_1$ *arg*$_n$** has the same semantics as "*cmd*_file *arg*$_1$ *arg*$_n$ **(at $‹arg›_0$)**", with the particular use of *arg*$_0$ as previously mentioned (in the definition of *cmd*_at). Subsequently, the latter command has the same semantics as "*cmd arg'*$_1$ *arg'*$_n$" with the resolving of files *arg*$_k$ performed as previously mentioned (in the definition of *cmd*_file).

10.3 Automatic Extraction of Promela Models from C11 Code

With a view to building Promela models of the SMP scheduler, some work has been done exploring how we might extract initial models from the C code directly, using the `modex` tool that is part of the Promela/SPIN toolsuite.

The automatic generation of C tests from Promela models assumed that we were starting from a state where we already conveniently got a Promela model at our disposal. Instead of writing this model by hand though, we are now interested to generate models from RTEMS source, so that one could rely on an automated mechanism to:

- get a high level of trusted computing base that the model we are relying on is formally close to the initial C code in input it is expecting to model,
- and generically have a way to apply the extraction process on the different relevant piece of RTEMS code; for instance the Scheduler or the Event Manager. This would also lower the maintenance effort on adapting any future change of the original C code to model.

Here we describe experiments done to help with analysing RTEMS C code, and particularly first focus on the RTEMS SMP scheduler as a case study.

As general picture, we plan to apply the SPIN model-extractor tool Modex, on the relevant C source of RTEMS dealing with the scheduler, to get a Promela model. The plan is to get initial Promela models that match the code closely, which we can then simplify and abstract in order to make analysis easier. These models will then be used, to generate tests for RTEMS, as described previously.

With a closer look at the Scheduler source, we notice it is using several features of C11 that can not be syntactically expressed in C99. Also the overall Scheduler source is fairly large, written in about 70k lines of code. In particular, a raw construction of a Promela model from the Scheduler source with Modex would be not immediately possible, because Modex can only accept code written in a subset of C99.

Model-checkers are only able to practically treat models up to a certain size (or associated number of automaton states). We will not treat the Scheduler source as a single file, but will mostly attempt to preserve its original folder structure, as it is currently found in the RTEMS source. This would permit us to quickly isolate those parts of the C source on which we should focus.

In this way, we will be able to delimit which sections of the Scheduler could be left behind an abstract black-box API, from those sections of real interest. Ideally, for each C file in the tree hierarchy of the RTEMS project at position `dir1/dir2/dir3/file1.c`, we would wish to obtain an exact Promela model situated at `dir1/dir2/dir3/file1.pml`. However, all the C files in the RTEMS project must first be preprocessed before one can apply Modex, as Modex requires a

closed ground source as input. This causes the first problem since the C preprocessing phase is concatenating all relevant C files into a single one. Indeed, applying Modex is not enough, as we also have to implement an inversion process to reconstitute back the initial tree hierarchy from the preprocessed C file. Additionally, since Modex is basically also returning a single Promela file, the same inversion process has to be applied on that Promela file.

Before going into deeper detail of how the preprocessing and tree reconstitution process functions, we present what we obtain as result in output.

- The full Scheduler source is automatically imported by `gcc` (in the preprocessing phase) by following the `#include`-dependencies of https://gitrepos.estec.esa.int/external/rtems-smp-qualification-rtems/-/blob/master/testsuites/smptests/smpmrsp01/init.c.

  In particular, the preprocessing result of `gcc` has been automatically arranged by our tool to be placed inside the directory `FV2-201_models/semaphore_manager/generated/smpmrsp01/raw/0_smpmrsp01/c`, abbreviated as `smpmrsp01_raw_c`, which contains:

  `/rsb` a folder replicating the structure of https://git.rtems.org/rtems-source-builder. In particular, `rsb/rtems/tmp/rtems/6` corresponds to the path set to contain all the binaries installed by the `rtems-source-builder` project, during its regular build invocation process.

  `/rtems` a folder replicating the structure of https://gitrepos.estec.esa.int/external/rtems-smp-qualification-rtems/-/tree/master/.

- Symmetrically, the full model of the Scheduler as generated by Modex (and post-treatment with our tools) is placed in `FV2-201_models/semaphore_manager/generated/smpmrsp01/raw/0_smpmrsp01/pml` (abbreviated as `smpmrsp01_raw_pml`). All files in this folder are Promela files, however their extensions do not finish with `.pml`, as instead, a `.c` or `.h` is used according to the extension of the original C file that this Promela model is expected to represent. For example, the Promela file `smpmrsp01_raw_pml/rtems/testsuites/smptests/smpmrsp01/init.c` is the model representing the C file `smpmrsp01_raw_c/rtems/testsuites/smptests/smpmrsp01/init.c`. We have explicitly kept all `.c` or `.h` extensions so that one can recursively apply graphical-difference-analysing tools to compare `smpmrsp01_raw_c` and `smpmrsp01_raw_pml`, such as `meld`[7] for example.

We now turn to the process generating `smpmrsp01_raw_c` and `smpmrsp01_raw_pml`. In a nutshell, this involves the following conversion steps, which are respectively described in the following sections:

- $\mathrm{C11} \rightarrow \mathrm{C}^{\mathrm{PP}}11$
- $\mathrm{C}^{\mathrm{PP}}11 \rightarrow \mathrm{C}^{\mathrm{PP}}99_{\mathrm{annot11}}$
- $\mathrm{C}^{\mathrm{PP}}99_{\mathrm{annot11}} \rightarrow \mathrm{C}^{\mathrm{PP}}99_{\mathrm{annot11}}{}^{\mathrm{CIL}}$
- $\mathrm{C}^{\mathrm{PP}}99_{\mathrm{annot11}}{}^{\mathrm{CIL}} \rightarrow \mathrm{C}^{\mathrm{PP}}99_{\mathrm{annot11+CIL}}{}^{\mathrm{ANSI}}$
- $\mathrm{C}^{\mathrm{PP}}99_{\mathrm{annot11+CIL}}{}^{\mathrm{ANSI}} \rightarrow \mathrm{PML}^{\mathrm{PP}}99_{\mathrm{annot11+CIL}}{}^{\mathrm{ANSI}}$
- $\mathrm{C}_{\mathrm{like}}99/11/18^{\mathrm{PP}} \rightarrow \mathrm{C}_{\mathrm{like}}99/11/18^{\mathrm{PP+TREE}}$:

  The last two $\mathrm{C}^{\mathrm{PP}}99_{\mathrm{annot11+CIL}}{}^{\mathrm{ANSI}}$ and $\mathrm{PML}^{\mathrm{PP}}99_{\mathrm{annot11+CIL}}{}^{\mathrm{ANSI}}$ respectively represent `smpmrsp01_raw_c` and `smpmrsp01_raw_pml`, but as a single `.c` file and `.pml` file. To additionally obtain a tree structure, we apply the generic function

[7] https://packages.debian.org/buster/meld

$C_{like}99/11/18^{PP} \rightarrow C_{like}99/11/18^{PP+TREE}$. This leads to $C^{PP+TREE}99_{annot11+CIL}{}^{ANSI}$ and $PML^{PP+TREE}99_{annot11+CIL}{}^{ANSI}$, now depicting `smpmrsp01_raw_c` and `smpmrsp01_raw_pml` in the two tree structures as found in `FV2-201_models/semaphore_manager/generated/smpmrsp01/raw/0_smpmrsp01`.

Note that the intermediate files are provided as http-links, as they are only provided here to illustrate the intermediate phase of the translation process producing `smpmrsp01_raw_c` and `smpmrsp01_raw_pml`. In contrast, the final folders `smpmrsp01_raw_c` and `smpmrsp01_raw_pml` are explicitly kept and saved in the repository.

10.3.1 Conversion Step Details

10.3.1.1 C11 → $C^{PP}11$

Setup of RTEMS and pre-processing the scheduler

From the output of "`./waf -v`", one can list all commands that are executed to compile the source of RTEMS. In particular, since the entry point of interest initializing the scheduler is situated in: `testsuites/smptests/smpmrsp01/init.c`, it is straightforward to get the full list of options given to `sparc-rtems5-gcc` to compile that file: `['sparc-rtems5-gcc', '-MMD', '-Wall', '-Wmissing-prototypes', '-Wimplicit-function-declaration', '-Wstrict-prototypes', '-Wnested-externs', '-O2', '-g', '-fdata-sections', '-ffunction-sections', '-mcpu=leon3', '-Icpukit / include', '-I.. / .. / .. / cpukit / include', '-Icpukit / score / cpu / sparc / include', '-I.. / .. / .. / cpukit / score / cpu / sparc / include', '-Ibsps / include', '-I.. / .. / .. / bsps / include', '-Ibsps / sparc / include', '-I.. / .. / .. / bsps / sparc / include', '-Ibsps / sparc / leon3 / include', '-I.. / .. / .. / bsps / sparc / leon3 / include', '-Itestsuites / support / include', '-I.. / .. / .. / testsuites / support / include', '.. / .. / .. / testsuites / smptests / smpmrsp01 / init.c', '-c', '-ortems / build / sparc / leon3 / testsuites / smptests / smpmrsp01 / init.c.566.o', '-DHAVE_CONFIG_H=1']`

However, instead of creating an object file, we can get a standalone pre-processed content, by replacing "-ortems / build / sparc / leon3 / testsuites / smptests / smpmrsp01 / init.c.566.o" with "-E" (and optionally followed by "-CC" to keep comments).

This leads to the respective files: https://www.scss.tcd.ie/frederic.tuong/init.c_pp and https://www.scss.tcd.ie/frederic.tuong/init.c_pp_CC.

10.3.1.2 $C^{PP}11 \rightarrow C^{PP}99_{annot11}$

Downgrading the C11 subset for Modex, towards $C99^{CIL}$ and $C99^{ANSI}$

The pre-processed obtained output is a very condensed file: composed of 69 kLoC (respectively 29 kLoC without comments), e.g. compared to the 27 kLoC of seL4 source (by counting comments).

To get a Promela model, one can try to apply Modex on the file. However, we obtain several parsing errors: the scheduler is implemented with several constructs not present in the C99 subset handled by Modex. Additionally, a quick check using parsers focusing on handling C11 and C18 constructs also reveals that it is accepted by the C11 parser we used[8], but not with the

[8] https://hackage.haskell.org/package/language-c

C18 one we used[9].

Noting what is said above, there are two solutions: taking advantage of an automated tool translating C11 to C99, or modifying by hand any part of the code with C11 constructs in order to temporarily disable them. Although the automated approach is the one we opt for a long term solution, one can actually also start by doing some statistics to measure the size of unsupported C11 constructs embedded in the initial input.

At the same time, one can also analyse the C99 parser used in Modex, to estimate the subset of C99 programs that Modex is supporting from the C99 standard. We actually notice its parser seems at first sight to accept less programs compared to other C99 parsers, e.g. such as the C99 default parser of Frama-C. Furthermore, a quick inspection of its source code reveals that Frama-C is using the CIL framework as an intermediate translation compilation phase: the CIL AST is a "generic" AST, simpler than the initial C99 AST. From the CIL framework, it is also possible to serialize to string and get in return a kind of "simplified" (subset) pre-processed C99 AST. In particular, we conjecture that the latter pre-processed C99 can be made acceptable to Modex more easily than raw C99 code not pre-processed by CIL.

Consequently, from the initial C11 AST, we temporarily disable by hand all C11 constructs rejected by the Frama-C parser (obviously changing at the same time the semantics of the scheduler). We obtain: https://www.scss.tcd.ie/frederic.tuong/init.c_pp0.

10.3.1.3 $C^{PP}99_{annot11} \rightarrow C^{PP}99_{annot11}{}^{CIL}$

Preprocessing with the CIL framework (included in Frama-C)

From this file, we apply the Frama-C parser to obtain a CIL representation. The latter serialization results in: https://www.scss.tcd.ie/frederic.tuong/init_out.c.

10.3.1.4 $C^{PP}99_{annot11}{}^{CIL} \rightarrow C^{PP}99_{annot11+CIL}{}^{ANSI}$

Downgrading the C99 subset for Modex

However, applying Modex on it is still not immediate, so we pursue additional manual preprocessing to obtain: https://www.scss.tcd.ie/frederic.tuong/init_out.c0.

10.3.1.5 $C^{PP}99_{annot11+CIL}{}^{ANSI} \rightarrow PML^{PP}99_{annot11+CIL}{}^{ANSI}$

Modex execution

Ultimately, applying Modex on the latter produces this Promela file: https://www.scss.tcd.ie/frederic.tuong/model. However it is rejected by spin, since this particular model-checking tool does not support a file with "too many channel types": `spin: model:3846, Error: too many channel types`.

[9] https://github.com/jhjourdan/C11parser (Note that besides C11, the project also supports C18, irrespective of its repository name.)

10.3.1.6 $C_{like}99/11/18^{PP} \rightarrow C_{like}99/11/18^{PP+TREE}$

Splitting a single preprocessed file into the original source tree hierarchy

This generation step is performed in `FV2-201_src/C_to_Promela/src/6_isabelle_c/src_split`. This uses Isabelle/HOL, and in particular, Isabelle/C, an implementation of C semantics in Isabelle/HOL.

10.3.2 Implementation and Statistics of the Sub-Generators

In this section, we show the implementation details of scripts and tools involved behind the different generation tasks. This will actually only concern tasks that are not in the category of heuristics, since these lasts mostly consist in producing their output by patching their input with some predefined replacement texts — even if, for the particular inputs we are currently considering, all replacements can be made automated and captured with regular expression patterns.

For the tasks that are automated, we compile a summarizing benchmark showing the overall performance of the tools for each respective task. Afterwards, we will analyze their minimal computing resources, which may highly vary depending on the different tools used.

We recall below the tasks that we are mainly focusing on:

- $C11 \rightarrow C^{PP}11$
  - Type of automation: complete (specialized to a first init.c file to consider)
  - Location: `FV2-201_src/C_to_Promela/src/1_gcc`
  - Abbreviation: gcc
- $C^{PP}11 \rightarrow C^{PP}99_{annot11}$
  - Type of automation: heuristic (not presented)
- $C^{PP}99_{annot11} \rightarrow C^{PP}99_{annot11}{}^{CIL}$
  - Type of automation: complete (in conjunction with heuristics preliminarily applied)
  - Location: `FV2-201_src/C_to_Promela/src/3_frama_c`
  - Abbreviation: frama_c
- $C^{PP}99_{annot11}{}^{CIL} \rightarrow C^{PP}99_{annot11+CIL}{}^{ANSI}$
  - Type of automation: heuristic (not presented)
- $C^{PP}99_{annot11+CIL}{}^{ANSI} \rightarrow PML^{PP}99_{annot11+CIL}{}^{ANSI}$
  - Type of automation: complete (in conjunction with heuristics preliminarily applied)
  - Location: `FV2-201_src/C_to_Promela/src/5_modex`
  - Abbreviation: modex
- $C_{like}99/11/18^{PP} \rightarrow C_{like}99/11/18^{PP+TREE}$
  - Type of automation: complete (generic translation, semantics not analyzed)

- Location: `FV2-201_src/C_to_Promela/src/6_isabelle_c`
- Abbreviation: isabelle_c

Even if each respective task is using as input the output result of the immediate previous task, we will run all of them independently in parallel from a same starting environment. This is to see for example which external packages are needed to be installed before running each tool. So it will require us to first generate all necessary input files, and retrieve (e.g. download) the appropriate input files at the beginning for each tool. In particular, we assume starting from a same new Debian[10] server (without any graphic supports) version 4.19.0-9-amd64 #1 SMP Debian 4.19.118-2+deb10u1 (2020-06-07) x86_64 GNU/Linux, installed using the minimal installation image https://cdimage.debian.org/debian-cd/current/amd64/iso-cd/debian-10.4.0-amd64-netinst.iso.

We also assume having at our disposal the packages `sudo` and `time` already installed, as well as `build-essential`, `dkms`, `linux-headers-4.19.0-9-amd64`. Note that these last packages (or their recursive dependent packages) might not be strictly needed by the tools. However, we explicitly installed them for efficiency reasons: we are actually making the experiments inside a virtual machine, which can take advantage of additional optimizations proper to the machine whenever these packages are installed.

Last, our virtual machine has a base memory of 8 GiB, a substantially same amount of SWAP memory, the possibility to concurrently use up to a fixed number of CPUs, and an initial free disk storage of at least 30 GiB. During the experiments, an effort was made to allocate the approximate same resources of the host machine to the guest machine while each task was running, especially the host CPU power, the host memory space, and the host network bandwidth (e.g. which might constantly change depending on the time of the day when the experiments are taken).

Location of the main generator script for each task

gcc

`FV2-201_src/C_to_Promela/src/1_gcc/full.sh`

(not presented)

`FV2-201_src/C_to_Promela/src/2_downgrade_c11/core.sh`

frama_c

`FV2-201_src/C_to_Promela/src/3_frama_c/full.sh`

(not presented)

`FV2-201_src/C_to_Promela/src/4_downgrade_c99/core.sh`

modex

`FV2-201_src/C_to_Promela/src/5_modex/full.sh`

isabelle_c

`FV2-201_src/C_to_Promela/src/6_isabelle_c/full.sh`

[10] Debian is the operating system that we primarily support in the rtems-smp-qualification project.

10.3.2.1 A Many-Core Setting with 6 CPUs

Time: Elapsed wall clock "minutes : seconds . and the first 10^{-2} digits"

gcc 31:42.93

frama_c 06:51.21

modex 00:23.96

isabelle_c 03:03.10

Disk: Increment usage

gcc 4,6 GiB

frama_c 2,3 GiB

modex 33 MiB

isabelle_c 1,6 GiB

Disk: Number of Debian packages additionally installed

gcc 074

frama_c 240

modex 014

isabelle_c 025

Network: Total received

gcc 324,12 MiB

frama_c 176,46 MiB

modex 005,94 MiB

isabelle_c 463,49 MiB

Network: Total sent

gcc 6,50 MiB

frama_c 3,53 MiB

modex 145 KiB

isabelle_c 8,70 MiB

Network: Average bitrate received

gcc 995,94 kbit/s

frama_c 03,12 Mbit/s

modex 01,47 Mbit/s

isabelle_c 18,00 Mbit/s

Network: Average bitrate sent

gcc 019,98 kbit/s

frama_c 062,51 kbit/s

modex 035,04 kbit/s

isabelle_c 337,85 kbit/s

Generator: Lines of code

gcc 56

frama_c 43

modex 34

isabelle_c 38

Generator: Number of shell top-level commands executed

gcc 40

frama_c 32

modex 14

isabelle_c 22

Additional GUI software provided

gcc The package does not include any GUI to build.

frama_c The *default* rules of installation perform the full build of the project, including the build of a GUI (and also a separate "command-line-only tool"), even on a server machine without graphical display.

modex The package does not include any GUI to build.

isabelle_c The package comes with a GUI (and also a separate "command-line-only tool"), but its building phase is by default only invoked the first time it is demanded (which was not executed in our case).

License

GCC

Usage

GPLv3+ with GCC Runtime Library Exception

No modifications were made on GCC.

Any other parts (e.g. those calling GCC or processing its output) that we modified are in BSD-3-Clause.

Frama-C

Usage

Parts of the components are either in CDDL-1.0, LGPL-2.0-or-later, LGPL-2.1-only, or Q modified license.

Modifications

LGPL-2.1-only is the licence of the parts that we modified.

Modex

Usage

All Rights Reserved. This software is for educational purposes only.

Permission is given to distribute the code provided that its introductory message is not removed and no monies are exchanged.

No modifications were made on Modex.

Any other parts (e.g. those calling Modex or processing its output) that we modified are in BSD-3-Clause.

Isabelle/C/Split

Usage

BSD-3-Clause

Modifications

BSD-3-Clause is the licence of the parts that we modified.

10.3.2.2 Same Configuration and Type of CPU as in Previous Experiments But Now With Only One Core

Time: Elapsed wall clock "hours : minutes : seconds . and the first 10^{-2} digits"

gcc 1:06:31.09

frama_c 0:11:15.45

modex 0:00:23.78

isabelle_c 0:04:06.52

Network: Average bitrate received

gcc 676,72 kbit/s

frama_c 02,16 Mbit/s

modex 01,56 Mbit/s

isabelle_c 15,31 Mbit/s

Network: Average bitrate sent

gcc 013,81 kbit/s

frama_c 042,98 kbit/s

modex 037,74 kbit/s

isabelle_c 286,47 kbit/s

10.3.2.3 Adjusted Elapsed Time Taking Into Consideration the Bandwidth Bitrates

The presented elapsed times included the time spent in making the download communication of the computer to the outside network, which performance may vary: e.g. depending on the length of files being downloaded, quantity of files to download, or server response time. Thus, the final time ranking might change.

However, as the results will show, we are going to not take into account the upload data, and only consider how download data are affecting the elapsed times. This is because for each task the size of upload data appears small, and occurs most often during the time range when a download is being made, or to be initiated closely soon after.

Globally, the overall ranking of the elapsed times of each task remains similar as previous results.

Time: Adjusted elapsed wall clock in the many-core setting

gcc 26:01.68

frama_c 05:51.90

modex 00:19.72

isabelle_c 02:36.10

Time: Adjusted elapsed wall clock in the mono-core setting

gcc 58:08.87

frama_c 10:16.15

modex 00:19.79

isabelle_c 03:34.78

10.3.2.4 Synthesis of Resource Usage

gcc

In this compiling task, we are only interested to the exact options given to GCC while compiling C files in the RTEMS source. Thus, it may be not necessary to perform the complete build of the project with GCC, which is triggered through `waf`. For example, one can imagine executing `waf` in a simulation mode to only print the list of commands it is planned to execute.

In more details, the build of the RSB repository could also be skipped, since it is occupying the major part of the task's time (25 min in the 6-cores experiments, and 55 min in the one core). Also, even if the build of a GCC cross-compiler would be fast, we are only interested to get a preprocessed version of a C file, which might after all be equally obtained from any other versions of GCC. In particular, we conjecture that the one already installed in the Debian machine would provide a same preprocessed file than the cross-compiled GCC (when both are called with the same GCC options).

Note that the cross-compiling phase appears to be nevertheless an unavoidable step, needed for other RTEMS-related projects, so the investment of effort and time spent in this task —while building a cross-compiler— will not be considered as wasted.

Overall, since this task is performing the worst on:

1. the total elapsed time,
2. the disk increment usage, and
3. the complexity of the generator or ease of installation (regarding lines of code and approximate number of top-level shell commands to execute),

then its benchmarks can serve as a minimum (acceptable) lower bound to not cross, for other tools of other tasks.

frama_c

The default installation of Frama-C triggers the complete build of the project which provides a GUI. There might be particular options to only trigger the build of its command line version, although which options to invoke do not appear to be immediate to find in the source code.

In parallel, the execution time of this task could be reduced by improving the number of packages installed by opam, as we are only targeting the command line version of the tool. In particular, several graphical packages (such as lablgtk) appears in the *mandatory* list of Frama-C's dependent packages, i.e. always installed before Frama-C. Similarly, removing these dependencies will impact in return the number of Debian packages to install, and lowering them.

Note that it is possible for opam to install several packages in parallel, a feature particularly useful in a many-core computing setting. However, there is a separation barrier between the code running the package manager and each code running their respective package installation. In particular, an instance of the latter might at some point be optimally in need of the resources of several cores, but there is no dynamic coordination between other computing instances regarding the number of cores available at a particular moment of the installation.

modex

Instead of implementing certain additional postprocessing out of Modex, the performance of this task would have been improved if the same modifications were directly implemented in it. However, we refrained from directly modifying Modex, as its license had been free only for a limited number of fields of endeavor.[11]

Practically, the benchmark shows that Modex's elapsed times do not substantially change when running it in a multi-core setting, compared to an equivalent but single-core experiment. Since a large part of the translation amounts to translating a C function to a respective Promela one, potentials for efficiently exploiting parallelism become higher whenever the number of C functions to translate is accordingly high (e.g. for the smpmrsp01 example, we currently have 821 functions). Note that this kind of exploitation is particularly feasible due to the fact that each C function to translate can be independently treated in parallel.

isabelle_c

A closer look at the downloaded data shows that it is the highest compared to other tools. Indeed, the archive of Isabelle2019 occupies a large part of the downloading

[11] https://www.debian.org/social_contract.html

activity: its size is 394.0 MiB. This archive contains several additional heap (binary) objects, that can all be removed to gain at least 73.1 MiB of compressed archive.[12] On the other hand, removing heaps is at the cost of making the tool's first invocation taking more CPU-time to reconstruct any missing ones, whenever really needed.

Still, not all heaps are required by Isabelle/C. Even if its source currently uses HOL as default entry-point, it is straightforward to rework its base dependencies, and replace any mentions of "HOL" by "Pure", its native ML kernel as equally valid entry-point.

Once HOL dependencies are removed, we can additionally drop its source from the archive (which are not used by Isabelle/Split), as well as the source of any unused components, such as its GUI (its build was not invoked) — e.g. one can put them in a separate archive for optional download.

Note that there is the possibility of dynamically downloading the internal package dependencies included in the archive, such as Java, Scala etc., through a script already provided in the developer repository. This configuration would actually be similar as a fresh clone from the Isabelle developer repository, where one starts with an archive without any pre-packaged binaries. Some of these binaries will always be mandatory to retrieve though (e.g. polyml). For the case of Isabelle2019, an archive version of its fresh clone would have a size of at most 17,8 MiB (and without counting yet the potential gain from the removal of unused source).

10.3.3 Conclusion

We have some powerful tools, based on Frama-C and Isabelle/C to allow us to process RTEMS sources in a variety of ways. These tools, however, are very heavyweight, and their use is unlikely to suit the majority or RTEMS users.

What they should help us to do, is to get the basic shape of suitable Promela models from RTEMS code, which we can simplify (carefully) so that model-checking, and hence, test-generation, become feasible.

The use of Frama-C in its own right may be something that we can advocate in the future, if we can address its codebase size.

10.4 Automatic Annotation Approach

The idea behind the automatic approach is that we develop "standard" Promela models that should describe correct API behaviour, along with appropriate use of `assert` and `ltl` statements to express desired properties. Once we are satisfied, via model-checking, that our model is correct, we then use automation from this point to generate the tests.

The automation involves parsing the Promela model file to obtain an abstract syntax tree of the model. An analysis phase identifies all instances of top-level behaviours, including `proctype` and `inline` declarations, and all `assert` and `ltl` statements. It then automatically adds in appropriate `printf` statements for the `proctype` (`inline` is similarly done) as part of the syntax

[12] When considering the full extracted archive, the gain amounts to 255.12 MiB.

tree. These `printf` statements identify the particular `proctype` being run, as well as outputting the actual runtime values of all of its arguments.

The generator then takes this modified AST and then, for each `assert` and `ltl` statement, considered individually, negates the relevant predicate (in a copy of the AST), and then pretty-prints the AST as a new Promela model file. There will be one such file for each `assert/ltl` statement.

Each of these is now model-checked, and should fail, generating a "counter-example" that corresponds to a correct run of the API.

This automatic system was developed independently from the manual one, and uses a different approach and notation. In Section 10.2, we present how we unified the two approaches, to some degree.

10.4.1 Model with Assertions

We start with a model in file `formal/promela/models/chains/src/cpukit.pml`, that focusses on the chain append API call.

```
inline append(ch,np) {
  assert(np!=0);
  // details elided
  assert(ch.size < 7);
  // details elided
}

proctype doAppend(int addr; int val) {
  atomic{
    memory[addr].itm = val;
    append(chain,addr);
  } ;
}
```

We have two assertions in `append` that ensure the model is valid: we treat array index 0 as a null C pointer; and we cannot have more than 7 elements in a chain.

We simply perform 3 appends (in parallel) and expect that the result will be a chain of length 3.

```
init {
  \\ details elided
  run doAppend(6,21);
  run doAppend(3,22);
  run doAppend(4,23);
  \\ details elided
  assert (chain.size == 3);
}
```

NOTE: material below needs to be revised.

We have three assertions, two associated with `append`, and one with `init`.

When the fully automatic test-generator is run, with the above model file (`cpukit.pml`), plus addition information discussed below, it will create three new subdirectories, `assert_0_append`, `assert_1_append` and `assert_init`. The model file in each will be called `init.pml`, and each will correspond to `cpukit.pml` with annotations automatically inserted. In addition, each will have one assertion or LTL predicate negated, in this example as follows (where `lno` is the line number of the original one in either `cpukit.pml` or `testsuites_spfreechain01.pml`):

Directory	lno	Original `cpukit.pml`	Negated `init.pml`
`assert_0_append`	123	`assert(np!=0)`	`assert(! (np!=0))`
`assert_1_append`	127	`assert(ch.size < 7)`	`assert(! (ch.size < 7))`
`assert_init`	70	`assert(chain.size == 3)`	`assert(! (chain.size == 3))`

From here on we will only consider the case of `assert_init/init.pml`, herinafter simply referred to as `init.pml`.

10.4.2 Automatic Annotations

We do not have any explicit annotations, but we still have to define a refinement relationship between the Promela model and C code.

The tool developed here is in fact one that takes a list of Promela sources as input, parses them, and then outputs the relevant generated Promela files.

However, in the examples developed for this, some of these Promela files simply contain one or more Promela comments, within which specific auto-generate commands are embedded. The commands cover a wide range of tasks, from what steps should be done, where outputs should go, details on how to run the SPIN model-checker, and details of how to convert counter-example output into C test code.

The file `formal/promela/models/chains/src/testsuites.cfg` has one Promela comment with configuration commands that specify the tasks to be done. We want to save generated Promela models, and human-readable counter-example (trail) files, in the export directory _.

```
export_pml \<open>init.pml\<close>
export_trail \<open>init.trail\<close>
export_dir \<open>_\<close>
```

The general format is of a command name (e.g. `export_dir`), followed by zero or more arguments, each delimited by `\<open>` and `\<close>`.

10.4.3 "Automatic" Refinement

We need to describe how to generate C test code for every "annotation point" in the Promela model. For now, an annotation point is just the declaration of a Promela process, using a `proctype` declaration.

In our example, we have on such `proctype`, called `doAppend`, defined on line 147 as follows:

```
proctype doAppend(int addr; int val) {
  atomic{
    memory[addr].itm = val;
    append(chain,addr);
  } ;
}
```

Note that `doAppend` takes two arguments, `addr` and `val`. What we want to do is to capture their exact values as the model runs, to use in tests.

In the corresponding refinement file `formal/promela/models/chains/src/testsuites_spfreechain01.rfn` we have just one command (`refine_uniform_at`) to generate the relevant C code.

```
refine_uniform_at \<open>doAppend\<close>
\<open>
#refine val0 @{promela [int] \<open>val\<close>}
#refine addr @{promela [int]}
    T_log(T_NORMAL,"@@@ 0 CALL append " xstr(val0) " " xstr(addr));
    memory[addr].val = val0;
    rtems_chain_append(&chain,(rtems_chain_node*)&memory[addr]);
\<close>
```

Here we use a special "refine" notation to build a link between a `proctype` parameter (`val`) and its value (`val0`). We shall describe how this is used in the next section. In the refinement file we also have a command that asks the tool to put a comment into the C file indicating which Promela `proctype` is being invoked:

```
refine_uniform_log
```

10.4.4 Automatic Annotation Generation

After parsing the original un-annotated model (`cpukit.pml`), the tool works through the assertions and LTL properties, negating just one at a time, as already described above.

The interesting example is the one where we negate the end assertion, that the final chain will have length 3, with the resulting generated Promela model shown in `assert_init/init.pml`.

That file has two modifications. The first is that the `doAppend` process now has a `printf` statement that outputs a unique key, as well as information about the arguments to the current call. The second argument *82* identifies *doAppend* by referring to its line number in the original *cpukit.pml* (without counting lines with comments).

```
active [0]  proctype doAppend(int addr; int val){
    atomic{
            memory[addr].itm = val
            append (chain, addr)
            printf("//@ 0.8444218515250481 \\<open>1\\<close> \\<open>82\\<close>
                    \\<open>%d\\<close> \\<open>%d\\<close>\n" , addr , val)
    }
}
```

(The `printf` string has line-breaks added to make viewing easier)

The second modification is that the 3rd `assert` has been negated (in the startup process `init`).

```
nr = _nr_pr
run doAppend (6 , 21)
run doAppend (3 , 22)
run doAppend (4 , 23)
(nr == _nr_pr)
assert((! (chain.size == 3)))
```

It now asserts (wrongly) that after three succesful `doAppend` calls, that the chain size will not be three.

10.4.5 Automatic Counterexample Generation

Another configuration file is `formal/promela/src/Promela_to_C/examples/model_checker/spin.pml`, which gives the command-line invocation for SPIN:

```
model_checker_verifier \<open>spin\<close> \<open>-a\<close>
model_checker_compile \<open>gcc\<close> \<open>-DVECTORSZ=4096\<close> \<open>-o\
↪<close> \<open>pan\<close> \<open>pan.c\<close>
model_checker_exec_one \<open>./pan\<close> \<open>-a\<close> \<open>-n\<close>
model_checker_exec_all \<open>./pan\<close> \<open>-a\<close> \<open>-n\<close> \
↪<open>-e\<close>
model_checker_trail \<open>spin\<close> \<open>-p\<close> \<open>-T\<close> \
↪<open>-t\<close> \<open>-k\<close>
```

This specifies that, given a generated Promela file `init.pml`, we should invoke SPIN in the following way:

```
spin -a init.pml
gcc -DVECTORSZ=4096 -o pan pan.c
./pan -a -n
spin -p -T -t -k init.pml.trail init.pml
```

When we run the above SPIN commands with the `init.pml` file described earlier, we obtain a file `init.trail`, which contains the counter-example it generated.

There are three such test-relevant lines. The first one is

```
//@ 0.8444218515250481 \<open>1\<close> \<open>82\<close> \<open>4\<close> \<open>
↪23\<close>
```

Here, `0.8444218515250481` is a hash-code that can be used to ensure that generated annotations can be traced back to specific files. This line describes a call to `doAppend` (File 1 `cpukit.pml`, line 82, without counting lines with comments) with arguments 4 and 23 for `addr` and `val` respectively. The other two test-relevant lines are very similar, with the only difference being the values for the last two arguments.

10.4.6 Automatic Code Generation

Given the three test-relevant lines from the trail file, we can now proceed to generate the test C code. The plans is to generate code for each of those lines in sequence, and place them between preamble and postamble C code, as was done in the manual code generation.

The pre-/post-amble files are defined by the first command in `testsuites_spfreechain01.rfn`, which is

```
enclose_file \<open>enclose_1_head.c\<close>
             \<open>enclose_2_foot.c\<close>
```

The generated test code is in `init.c`.

The trail file line

```
//@ 0.8444218515250481 \<open>1\<close> \<open>82\<close> \<open>4\<close> \<open>
↪23\<close>
```

results in the following (concrete) C code:

```
//@ promela_term \<open>doAppend\<close>
#define val0 23
#define addr 4
    T_log(T_NORMAL,"@@@ 0 CALL append " xstr(val0) " " xstr(addr));
    memory[addr].val = val0;
    rtems_chain_append(&chain,(rtems_chain_node*)&memory[addr]);
#undef val0
#undef addr
```

When the code is compiled and run, we get file `init.log`.

The most interesting part of `init.log` is:

```
L:@@@INIT
L:doAppend(addr=4,val=23)
P:0:0:UI1:init.c:117
P:1:0:UI1:init.c:118
L:doAppend(addr=3,val=22)
P:2:0:UI1:init.c:126
P:3:0:UI1:init.c:127
L:doAppend(addr=6,val=21)
P:4:0:UI1:init.c:135
P:5:0:UI1:init.c:136
```

10.4.7 "Automatic" Software Tools

10.4.7.1 testgen

Currently, this fully automatic generator, `testgen` is written in "Coconut", a python extension that supports some functional programming styles, including pattern-matching. It is invoked via a bash-script `formal/promela/src/testgen_ml_spin.sh`, which compiles Coconut (`formal/promela/src/src/testgen.coco`) into standard python (`testgen.py`), which is then executed to do the test generation.

For now, we have the following directory structure, rooted at `formal/promela/src/`.

`/document` Local documentation.

- **`/source`** Technical documentation in RST.

`/examples` Examples directory.

- **`/model_checker/spin.pml`** File included by `testgen_ml_spin.sh` that gives commands to run SPIN to find a counter-example.

`/src` Test Generator sources

- **`testgen_ml.sh`** Generic bash script.
- **`testgen_ml_spin.sh`** Tailoring of `testgen_ml.sh` to run test generator.

`/src/src/modules` 3rd-party Python modules

- **`/comment_filter`** Code to pull comments out of source files, with code replaced by blanks.
- **`/promela_yacc`** A Promela parser written in python.

The best way to run the test generation example in this document at present is as follows (**Needs Revising**):

```
cd formal/promela/models/chains
../../../src/Promela_to_C/src/testgen_ml_spin.sh testsuites_spfreechain01.pml␣
↪testsuites_spfreechain01.rfn --promela "$(cat testsuites.cfg)"
```

When this has completed, a sub-directory _ will be created with the following contents:

/assert_0_append Result of negating first doAppend assert statement.

/assert_1_append Result of negating second doAppend assert statement.

/assert_init Result of negating the only `init` assert statement.

Each of the above directories contains three files:

init.pml The Promela file with annotations automatically added

init.trail The resulting counter-example output.

init.c or generic/init.c or uniform/init.c The resulting C test program, using the new RTEMS Testing Framework Release 6. This has succesfully been built using `waf`, and run using `sparc-rtems6-sis`.

10.5 Automatic Approach: Modelling of the Event Manager in Promela

10.5.1 Overview

This is out of date: files have been moved around, needs updating. It also predates the introductions of model-0.yml and all its associated infrastructure.

In this section, we provide a model of the Event Manager. This model has the characteristic of being as close as possible to the real C11 code. It is not derived from the human readable documents accompanying the RTEMS source, rather it is derived based on the understanding that one might possibly have of the code.

Our version of the Event Manager is situated in `formal/promela/models/events/src` (where `FV2-201_models` is the abbreviation we give to the first directory of the git repository having models as name, and `FV2-201_models/events` is abbreviated as `models_events`), and particularly in `models_events/src/cpukit.pml` containing the core implementation in Promela (representing `cpukit/rtems/src/eventsurrender.c` and `cpukit/rtems/src/eventseize.c`). The directory includes several examples:

- `models_events/src/testsuites1.pml` covers random cases of sending and receiving actions of events. Situations of deadlocks are allowed and can occur for instance when a receiving action remains blocked without a corresponding send action that can satisfy it.
- `models_events/src/testsuites2.pml` is similar to `models_events/src/testsuites1.pml` but any receiving actions are guaranteed to be ultimately not pending. After a number of iteration steps (either a send or receive action), we analyze if there is any receive actions not fulfilled. If this is the case, `models_events/src/testsuites2.pml` triggers the appropriate send actions to force the resuming of blocked processes. Additionally, in contrast to the `models_events/src/testsuites1.pml` example, this model is proved to be always correctly terminating by the model-checker.

Inside the directory, one can also find the three instancing examples of `testsuites/sptests/spintrcritical10/init.c` now independently split and respectively modelled in three different files:

- `models_events/src/testsuites_spintrcritical10_1.pml`

- `models_events/src/testsuites_spintrcritical10_2.pml`
- `models_events/src/testsuites_spintrcritical10_3.pml`

After executing the following script (linebreaks added here for clarity),

```
formal/promela/src/Promela_to_C/src/testgen_spin.sh
      models_events/src/testsuites_spintrcritical10_1.pml
      models_events/src/testsuites_spintrcritical10_1.rfn
      --promela "$(cat models_events/src/testsuites.cfg)"
```

in some directory (where `FV2-201_src` is the abbreviation we give to the first directory of the git repository having src as name), a new folder, simply called _, gets generated containing, among other things, our C output file of interest (that folder location had been specified in `models_events/src/testsuites.cfg`).

For convenience, we manually provide, in `models_events/generated` , the output folder _ of what would have been generated by the above script, so that one can inspect it without the need to run the script.

This gives us, for example, our desired C test file `models_events/generated/src/testsuites_spintrcritical10_1.auto_output/ltl_termination/init.c`. Recall that part of that file is precisely dependent on the content of our provided refinement file `models_events/src/testsuites_spintrcritical10_1.rfn`.

To run this `init.c` file with the correct compilation options, one can take advantage of the build program `waf`, provided in RTEMS 6. For example, we can manually overwrite any `init.c` in the `testsuites` directory (chosen randomly), such as this one: `testsuites/sptests/spintrcritical10/init.c`.

On a machine having `$HOME/tmp/rsb/6/bin` as directory set up by "rtems-source-builder" (https://git.rtems.org/rtems-source-builder), it becomes then straightforward to run the `sparc-rtems6-sis` simulator using:

```
export PATH=$HOME/tmp/rsb/6/bin:"$PATH"
./waf -v
sparc-rtems6-sis -leon3 -r build/sparc/leon3/testsuites/sptests/spintrcritical10.
↪exe
```

10.5.2 Architecture Model

Our implementation of send and receive in `models_events/src/cpukit.pml` is pretty close to the C RTEMS source: we only have two Promela proctypes `event_surrender_exec` (representing send) and `event_seize_exec` (representing receive) that might be asynchronously launched at any time. However, the main starting points for end-users are respectively located in `event_surrender_send` and `event_seize_receive`.

Technically, our implementation model can even be slightly more general than the C implementation in RTEMS. For instance, our `event_surrender_send` can take two process identifiers for the origin thread and destination thread, whereas in the C implementation, a sending action is assumed to originate from a "self" thread.

The C implementation is highly concurrent: a send can happen at the same time than a receive. This makes our test generation to be slightly different than what we modelled for the scheduler. Indeed, our auto approach expects to associate exactly one printf statement with only one proctype. However here, it would be possible to have several atomic blocks inside one proctype.

In the end, while running our auto approach, we would be not able to do a fine-grain identification of which atomics inside a particular proctype are produced in the trail trace, unless we change and adapt the C source so that we have at most one C function executing an atomic action in Promela. This is something we have not performed; and especially the reason why we inserted a new option `no_atomic` in `models_events/src/testsuites_spintrcritical10_1.rfn` to disable the generation of any atomic statements.

CHAPTER

ELEVEN

REFERENCES

BIBLIOGRAPHY

[BH19] Andrew Butterfield and Mike Hinchey. *Formal Verification Plan*. 2019.

[Jaskuc22] Jerzy Jaśkuć. SPIN/Promela-Based Test Generation Framework for RTEMS Barrier Manager. Master's thesis, School of Computer Science and Statistics, Trinity College, Dublin 2, Ireland, April 2022.

[Lyn22] Eoin Lynch. Using Promela/SPIN to do Test Generation for RTEMS-SMP. Master's thesis, School of Engineering, Trinity College, Dublin 2, Ireland, April 2022.

[UAHB10] Andreas Ulrich, El Hachemi Alikacem, Hesham Hallal, and Sergiy Boroday. From scenarios to test implementations via promela. In Alexandre Petrenko, Adenilso da Silva Simão, and José Carlos Maldonado, editors, *Testing Software and Systems - 22nd IFIP WG 6.1 International Conference, ICTSS 2010, Natal, Brazil, November 8-10, 2010. Proceedings*, volume 6435 of Lecture Notes in Computer Science, 236–249. Springer, 2010. URL: https://doi.org/10.1007/978-3-642-16573-3_17, doi:10.1007/978-3-642-16573-3_17.